UNIWERSYTET KARDYNAŁA STEFANA WYSZYŃSKIEGO
W WARSZAWIE

WYDZIAŁ MATEMATYCZNO-PRZYRODNICZY
SZKOŁA NAUK ŚCISŁYCH

Kacper Smykla

Nr alb.: 123627

Kierunek studiów: informatyka

Zastosowanie metod głębokiego uczenia maszynowego do klasyfikacji ruchu sieciowego jako szkodliwego lub nieszkodliwego

Praca licencjacka
Promotor:
dr inż. Robert A. Kłopotek

WARSZAWA 2025

Załącznik do Zarządzenia Nr 78/2014 Rektora UKSW
z dnia 14 listopada 2014 r.
Załącznik nr 3 do Zarządzenia Nr 39/2007 Rektora UKSW
z dnia 9 listopada 2007r.

……
Imię i nazwisko studenta/studentki
……
Nr albumu
……..
Wydział
……
Instytut
……..
Kierunek

Dziekan Wydziału ……..

……..

OŚWIADCZENIE

Świadomy(a) odpowiedzialności prawnej oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Oświadczam również, że przedstawiona praca nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w żadnej uczelni.

Oświadczam ponadto, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną.

Oświadczam, że poinformowano mnie o zasadach dotyczących kontroli samodzielności prac dyplomowych i zaliczeniowych. W związku z powyższym oświadczam, że wyrażam zgodę na przetwarzanie* moich prac pisemnych (w tym prac zaliczeniowych i pracy dyplomowej) powstałych w toku studiów i związanych z realizacją programu kształcenia w Uczelni, a także na przechowywanie pracy dyplomowej w celach realizowanej procedury antyplagiatowej w ogólnopolskim repozytorium pisemnych prac dyplomowych.

…….....
podpis studenta

*Przez przetwarzanie pracy rozumie się porównywanie przez system antyplagiatowy jej treści z innymi dokumentami (w celu ustalenia istnienia nieuprawnionych zapożyczeń) oraz generowanie raportu podobieństwa.

OŚWIADCZENIE

Oświadczam, że niniejsza praca napisana przez

Pana/Panią....., nr albumu ..... została przygotowana pod moim kierunkiem i stwierdzam, że spełnia ona warunki do przedstawienia jej w postepowaniu o nadanie tytułu zawodowego.

…….....
podpis promotora

Załącznik do Zarządzenia nr 14/2010
Rektora UKSW z dnia 15 marca 2010 r.
Załącznik nr 4
do Zarządzenia nr 39/07
Rektora UKSW z dnia 9 listopada 2007r.

Imię i nazwisko autora pracy	
Imię i nazwisko promotora pracy	
Wydział	
Kierunek studiów	
Specjalność	
Tytuł pracy	

Niniejszym oświadczam, że zachowując moje prawa autorskie, udzielam Uniwersytetowi Kardynała Stefana Wyszyńskiego w Warszawie nieodpłatnej licencji niewyłącznej do korzystania z ww. pracy bez ograniczeń czasowych w zakresie rozpowszechniania pracy poprzez publiczne udostępnianie pracy w wersji drukowanej i elektronicznej , w taki sposób, aby każdy zainteresowany mógł mieć do niej dostęp w miejscu, w którym praca jest przechowywana tj.: w Archiwum UKSW lub w Bibliotece UKSW.

/podpis /

Spis treści

1. Wstęp . . . 11
1.1. Motywacja . . . 11
1.2. Cel pracy . . . 12

2. Przedstawienie problemu badawczego . . . 13
2.1. Klasyfikacja pakietów w kontekście systemów obronnych . . . 13
2.2. Rodzaje systemów IDS . . . 14
2.3. Rosnąca skala zagrożeń a typy ataków . . . 15
2.4. Sposób przetwarzania pakietów a wybór cech . . . 15
2.5. Problemy dotyczące danych wejściowych . . . 17
2.6. Problem fałszywych pozytywów i negatywów . . . 18
2.7. Architektura modelu hybrydowego . . . 19
2.7.1. Sposób wczytywania i otrzymywania danych . . . 19
2.7.2. Opis działania sieci neuronowych . . . 20
2.7.3. Opis działania sieci konwolucyjnych . . . 22
2.7.4. Opis działania sieci rekurencyjnych . . . 25
2.7.5. Opis warstw modelu i ich parametrów . . . 25

3. Implementacja modelów i porównanie . . . 27
3.1. Opis porównywanych modeli i parametrów . . . 27
3.2. Opis programów realizujących modele SI . . . 31
3.3. Opis środowiska programu potrzebnego do uruchomienia . . . 33
3.4. Wykaz funkcji głównych i pomocniczych wykorzystywanych w programach . . . 35
3.5. Uruchomienie programu i odczytywanie informacji zwrotnych . . . 36

4. Metodologia badań . . . 37
4.1. Pytania pomocnicze . . . 37
4.2. Metryki wykorzystane do pomiarów wydajności . . . 38
4.3. Warunki testowe . . . 39

5. Wyniki eksperymentów . . . 41

6. Podsumowanie . . . 45

Bibliografia . . . 47

Spis rysunków

3.1. Wykaz 12 najważniejszych cech modelu RF 30
3.2. Wykaz 12 najważniejszych cech modelu SVM 30
3.3. Schemat przepływu danych i modułów w systemie eksperymentalnym 32

Spis tabel

2.1. Macierz Pomyłek (Confusion Matrix) dla Klasyfikacji Binarnej 18

3.1. Porównanie parametrów modelu hybrydowego wraz z jego wariantami 31

5.1. Skuteczność modeli na danych testowych 41
5.2. Skuteczność modeli na danych treningowych 41

Rozdział 1

Wstęp

1.1. Motywacja

Współczesne sieci komputerowe, funkcjonujące w dynamicznym środowisku, jakim jest Internet, są szczególnie narażone na zagrożenia ze strony hakerów. Problem włamań do systemów i sieci komputerowych istnieje od wielu lat, jednakże wraz ze stopniową cyfryzacją kluczowej infrastruktury wzrasta ryzyko dla instytucji państwowych oraz obywateli, których dane są tam przechowywane. Świat biznesu również adaptuje się do nowej technologii, wdrażając rozwiązania usprawniające pracę, w tym narzędzia oparte na sztucznej inteligencji, takie jak duże modele językowe. Ponieważ zarówno instytucje państwowe, większość obywateli, jak i sektor prywatny tak silnie polegają na infrastrukturze IT, szczególnego znaczenia nabiera zapewnienie bezpieczeństwa użytkowników oraz stabilności tej infrastruktury.

Zagadnienie bezpieczeństwa informatycznego jest również kluczowe dla osób projektujących i wdrażających systemy IT. W obecnym klimacie geopolitycznym państwa zwracają coraz większą uwagę na ochronę danych osobowych obywateli oraz zabezpieczenie infrastruktury cyfrowej. W odpowiedzi na te wyzwania powstały liczne regulacje, takie jak unijne dyrektywy NIS, NIS2, GDPR (RODO w Polsce) czy Cyber Resilience Act, które nakładają surowe kary finansowe za niewłaściwe zabezpieczenie danych i infrastruktury. Dane osobowe, które często przetwarzane są przez instytucje publiczne i prywatne, stają się motywacja dla hakerów do włamywania się do systemu, gdyż zawsze są cenne i zawsze można je łatwo sprzedać, co stanowi ryzyko dla każdej jednostki przetwarzającej dane. Jeszcze większe ryzyko pojawia się przy świadczeniu usług online, takich jak strony internetowe czy serwery. W takim kontekście potencjalny haker może nie tylko wykradać dane, ale także próbować zakłócić działanie usług, na przykład za pomocą ataków DDoS, które mogą zatrzymać lub znacząco spowolnić kluczowe usługi IT, narażając ich dostawców na wysokie straty finansowe.

Systemy wykrywania włamań (Intrusion Detection Systems - IDS) stanowią pierwszy krok w przeciwdziałaniu atakom, zapewniając administratorom oraz innym systemom zapobiegającym włamaniom (Intrusion Prevention Systems - IPS) możliwości szybkiego ostrzegania oraz bazę wiedzy nt włamań. Umożliwia to wcześniejsze podejmowanie działań zabezpieczających i skuteczniejszą ochronę infrastruktury.

Tworzenie zabezpieczeń oraz rozwój nowych metod ataków to procesy wzajemnie napędzające się, tworzące nieustanny cykl innowacji i kontrataków. Podobnie jak w wyścigu zbrojeń, każde nowe zabezpieczenie wywołuje reakcję po stronie osób próbujących przełamać ochronę, co prowadzi do coraz bardziej zaawansowanych technik ataków. Z kolei wykrycie i analiza tych ataków zmusza twórców systemów zabezpieczeń do opracowywania coraz skuteczniejszych narzędzi ochrony. Ta dynamiczna relacja oznacza, że zarówno środki obronne, jak i ofensywne stale się rozwijają i doskonalą, aby zyskać przewagę nad przeciwną stroną. W rezultacie przemysł cyberbezpieczeństwa jest nieustannie

stymulowany do postępu, a obie strony wyścigu technologicznego próbują przewidzieć i wyprzedzić działania drugiej strony, co skutkuje rozwojem bardziej kompleksowych rozwiązań.

Rosnąca złożoność i różnorodność nowych form ataków wymusza zastosowanie sztucznej inteligencji (SI) jako narzędzia do efektywnego dostosowania się do zmieniających się zagrożeń, które coraz częściej przewyższa możliwości tradycyjnych metod opartych na regułach i ręcznych aktualizacjach.

Algorytmy AI mogą analizować ogromne ilości danych w czasie rzeczywistym, wykrywać subtelne anomalie w zachowaniu ruchu sieciowego oraz dostosowywać się do nowych rodzajów zagrożeń bez konieczności manualnej aktualizacji reguł. Wykorzystując metody uczenia maszynowego i głębokiego, systemy oparte na SI potrafią uczyć się na podstawie poprzednich zdarzeń, umożliwiając proaktywną ochronę i adaptację do ciągle ewoluującego środowiska zagrożeń.

1.2. Cel pracy

Celem niniejszej pracy jest opracowanie hybrydowego modelu sztucznej inteligencji do klasyfikacji ruchu sieciowego jako szkodliwego lub nieszkodliwego. Model będzie analizował pakiety odczytywane z plików w formacie pcap, który stanowi standard przechowywania danych dotyczących ruchu sieciowego. Rdzeniem modelu jest połączenie konwolucyjnej sieci neuronowej (CNN) z długoterminową pamięcią krótkotrwałą (LSTM), co umożliwi analizę zarówno czasowych, jak i przestrzennych zależności między pakietami.

W ramach pracy postawiona zostanie hipoteza badawcza: klasyczne algorytmy uczenia maszynowego są niewystarczające do skutecznej klasyfikacji ruchu sieciowego w warunkach dużej złożoności danych, a zastosowanie hybrydowych architektur głębokiego uczenia (np. CNN + LSTM) pozwala na osiągnięcie wyższej skuteczności wykrywania ruchu szkodliwego.

Rozdział 2

Przedstawienie problemu badawczego

Celem niniejszej pracy jest zaprojektowanie hybrydowego modelu detekcji szkodliwych pakietów na podstawie ruchu sieciowego. Stawiam tezę, że połączenie konwolucyjnych i rekurencyjnych sieci neuronowych pozwala na lepsze wykrywanie ataków oraz poprawę ogólnej skuteczności systemu IDS. Problematyka ta nabiera szczególnego znaczenia w obliczu dynamicznie zmieniającego się krajobrazu zagrożeń cybernetycznych, gdzie tradycyjne metody przestają być wystarczające w detekcji zarówno znanych, jak i nowych typów ataków [10]. Temat ten jest silnie związany z tematyką wykrywania zagrożeń i systemów wykrywania włamań (IDS).

W dalszych sekcjach omówiono będzie szerszy kontekst i podobieństwo do projektowania systemów detekcji intruzji (IDS) oraz wyzwań związanych z ich funkcjonowaniem. Przedstawiono klasyfikację pakietów w kontekście systemów obronnych, rodzaje systemów IDS oraz analizę wpływu rosnącej skali zagrożeń i typów ataków na efektywność tych systemów. Omówiono również znaczenie sposobu przetwarzania pakietów w kontekście wyboru kluczowych cech oraz problemy związane z jakością danych wejściowych. Szczególną uwagę poświęcono kwestiom fałszywych pozytywów i negatywów oraz ich wpływowi na skuteczność detekcji. W części dotyczącej architektury modelu hybrydowego szczegółowo opisano działanie sieci konwolucyjnych, rekurencyjnych oraz ich wariantu LSTM, wraz z procesem doboru parametrów i optymalizacji modelu.

2.1. Klasyfikacja pakietów w kontekście systemów obronnych

Współczesne systemy bezpieczeństwa sieciowego charakteryzują się wysokim stopniem złożoności i różnorodnością, obejmując różne typy narzędzi i technologii ochronnych. Kluczowymi elementami tej infrastruktury są systemy SIEM (Security Information and Event Management), IDS (Intrusion Detection Systems) oraz IPS (Intrusion Prevention Systems).

Systemy IDS służą do wykrywania potencjalnych zagrożeń w ruchu sieciowym i zgłaszania ich odpowiednim jednostkom, najczęściej administratorom. Choć efektywnie identyfikują anomalie, wymagają ręcznej interwencji w celu neutralizacji zagrożeń, co ogranicza ich autonomię działania.

Problem ten rozwiązują systemy IPS, które oprócz wykrywania zagrożeń automatycznie podejmują działania prewencyjne, takie jak blokowanie ruchu sieciowego pochodzącego z podejrzanych źródeł. Oba typy systemów często funkcjonują komplementarnie, stanowiąc elementy bardziej zintegrowanych i kompleksowych rozwiązań ochronnych.

Systemy SIEM, takie jak Wazuh, zapewniają bardziej holistyczne podejście do zarządzania bezpieczeństwem. Integrują logi z urządzeń i usług całej organizacji, umożliwiając korelację zdarzeń i zaawansowaną analizę zagrożeń w celu podniesienia poziomu bezpieczeństwa.

W tym kontekście proponowany przeze mnie model może być traktowany jako potencjalny wyspecjalizowany komponent systemu IDS, jako jego wąski podzbiór, szczególnie działający w dziedzinie

ruchu sieciowego. Ponieważ proponowany przez mnie model jest silnie powiązany z tematyką bezpieczeństwa i systemów do wykrywania włamań w poniższych sekcjach będę rozważał problemy typowe dla systemów IDS i próbował je odnieść do mojego modelu, znaleźć podobieństwa i wspólne problemy do rozwiązania.

2.2. Rodzaje systemów IDS

Systemy IDS (Intrusion Detection Systems) dzielą się na dwie główne kategorie [10]: oparte na regułach oraz bazujące na wykrywaniu anomalii. Rozwiązania oparte na regułach (signature-based) wykorzystują zbiory zdefiniowanych wcześniej reguł, które umożliwiają skuteczne wykrywanie wielu znanych typów ataków. Są one powszechnie stosowane i dobrze radzą sobie z identyfikacją zagrożeń, które zostały wcześniej zidentyfikowane i opisane w bazie. Główną wadą tej techniki jest jednak niekompletność tych baz. W dynamicznie zmieniającym się środowisku cyberbezpieczeństwa nowe zagrożenia pojawiają się często, a stare luki bezpieczeństwa są regularnie eliminowane dzięki aktualizacjom oprogramowania. Hakerzy muszą więc albo wykorzystywać przestarzałe i nienaprawione oprogramowanie, albo tworzyć nowe, wcześniej nieznane luki, które nie są jeszcze nieznane, a za tym nie objęte żadnymi regułami.

Rozwiązania oparte na regułach mogą nie zidentyfikować nowych rodzajów ataków, ponieważ reguły dla takich zagrożeń po prostu nie istnieją. Ponadto konieczność ręcznego uzupełniania lub udoskonalania reguł w odpowiedzi na nowe zagrożenia może prowadzić do opóźnień w ochronie. Choć systemy te generują stosunkowo niewiele fałszywych pozytywów w porównaniu z podejściem opartym na anomaliach, jakość reguł ma kluczowe znaczenie. Zbyt restrykcyjne filtry albo niepoprawnie sformułowane zasady mogą prowadzić do pogorszenia jakości wykrywania i licznych błędów a w szczególności fałszywych pozytywów.

Z kolei systemy IDS bazujące na anomaliach (anomaly-based) koncentrują się na wykrywaniu odchyleń od normalnego wzorca ruchu sieciowego. Tego typu podejście częściowo rozwiązuje problem wykrywania nowych zagrożeń, ponieważ systemy te mogą identyfikować nieznane wcześniej ataki na podstawie ich nietypowego zachowania. Jednakże, główną wadą tego podejścia jest znaczny wzrost liczby fałszywych pozytywów. Warto też pamiętać, że nie każde odchylenie od normy oznacza atak, co może prowadzić do błędnych alarmów i obniżenia efektywności. Dodatkowo, systemy bazujące na anomaliach są szczególnie wrażliwe na jakość danych używanych do ich trenowania, co może prowadzić do przyuczania się i zmniejszonej efektywności.

Pomimo swoich ograniczeń, systemy bazujące na anomaliach są istotnym uzupełnieniem podejścia opartego na regułach, szczególnie w dynamicznie zmieniających się środowiskach, gdzie szybkie reagowanie na nowe i nieprzewidywalne zagrożenia ma kluczowe znaczenie. Połączenie obu technik w hybrydowych modelach IDS pozwala na lepsze wykorzystanie zalet obu podejść, minimalizując ich wady [11].

Proponowany przeze mnie model hybrydowy nie wpisuje się jednoznacznie w tradycyjne kategorie systemów IDS opartych na regułach lub anomaliach. Model ten nie definiuje ani nie przechowuje reguł w postaci czytelnej przez człowieka składni, które mogłyby być zapisane w jakiejś bazie. Niemniej jednak, w procesie uczenia model może nauczyć się identyfikować pewne nieprawidłowości w danych, powiązane z konkretnymi wartościami bajtów w pakietach, co w ograniczonym zakresie może przypominać identyfikację opartą na regułach. Jednakże brak czytelnego zapisu reguł w postaci jakiejś struktury lub składni ogranicza możliwość ich ręcznego przeglądu, modyfikacji czy dodawania wyjątków, co stanowi istotną przewagę tradycyjnych podejść opartych na regułach.

Z drugiej strony, skuteczność modelu w wykrywaniu anomalii również budzi wątpliwości. Zdolność modelu do wykrywania anomalii opiera się na porównywaniu badanego zachowania z uprzednio zdefiniowaną „normą” – wzorcem uznanym za reprezentatywny dla ruchu nieszkodliwego. Jeśli ten

wzorzec jest niedokładny lub niepełny, model nie będzie w stanie wiarygodnie odróżniać odchyleń od typowego zachowania, co prowadzi do pogorszenia się wskaźników takich jak dokładność, czułość czy precyzja. Jednocześnie, brak odpowiednio reprezentowanych przypadków anomalii w danych treningowych uniemożliwia modelowi wyuczenie charakterystyk odchyleń, a tym samym skuteczne rozpoznanie nowych zagrożeń. Istotną przeszkodą jest fakt, że powszechnie stosowane zbiory danych dla systemów IDS/IPS nie oddają w pełni rzeczywistego spektrum ataków [4], przez co definicja „normy" bazuje na ograniczonym i często niereprezentatywnym materiale. W rezultacie modele uczone na takich danych są narażone na niedoszacowanie lub wręcz pominięcie nietypowych, lecz krytycznych zdarzeń.

Ostateczna efektywność w tym zakresie zależy od zdolności modelu do generalizacji, czyli rozpoznawania wzorców, które nie były bezpośrednio widziane podczas treningu. W związku z tym istotnym pytaniem badawczym jest, czy integracja metod dodatkowego wykrywania anomalii – takich jak autoenkodery lub techniki klastrowania – mogłaby zwiększyć skuteczność modelu w identyfikacji nieznanych zagrożeń. Jednocześnie kluczowym wyzwaniem pozostaje zminimalizowanie liczby fałszywych pozytywów, aby utrzymać praktyczną użyteczność systemu.

2.3. Rosnąca skala zagrożeń a typy ataków

Systemy IDS aby skutecznie działać w rzeczywistym środowisku, muszą uwzględniać różne typy ataków możliwe do przeprowadzenia przez hakerów. Typowe rodzaje ataków to [4] m.in. U2R (User-to-Root), R2L (Remote-to-Local), DoS (Denial-of-Service), DDoS (Distributed Denial-of-Service), scanning/probing oraz brute force.

Ataki U2R polegają na eskalacji uprawnień użytkownika do dostępu administratora. Choć niezwykle groźne, ich wykrycie na poziomie pakietów sieciowych jest trudne, ponieważ dotyczą one zwykle działania programów lub aplikacji po uzyskaniu dostępu. Bardzo mało prawdopodobne jest więc, aby mogły zostać wykryte przez modele AI skanujące ruch sieciowy.

Ataki R2L, takie jak phishing czy próby przechwycenia sesji, polegają na uzyskaniu nieautoryzowanego dostępu do systemu zdalnie poprzez wykorzystanie podatności, pozwalającej na zdalny dostęp. O ile wykrycie luki umożliwiającej taki dostęp będzie ciężkie z perspektywy modelu analizujący ruch sieciowy, to jest szansa że można będzie wykryć taką klasę ataku poprzez wykrycie towarzyszących temu atakowi innych towarzyszących znaków. Charakterystyczną cechą są nietypowe wzorce logowania czy nadmiarowe połączenia do reverse shell zostawionego w przypadku takiego ataku w celach eksploatacji ofiary po skutecznym ataku, co proponowany model powinien móc wykrywać dzięki analizie wzorców czasowych i analizie potencjalnie anomalnych zachowań.

Najczęstszymi zagrożeniami są jednak ataki typu DoS i DDoS, które mają na celu przeciążenie systemu i zakłócenie dostępności usług. Ataki DoS i DDoS powinny zostać wykryte przez model hybrydowy. Kolejnym typem są scanning/probing, gdzie haker bada sieć w celu wykrycia otwartych portów lub podatnych usług. Ataki te można wykryć, analizując powtarzalne próby nawiązywania połączeń z różnymi adresami lub portami w krótkim czasie, co jest doskonale uchwytne dzięki analizie sekwencji czasowych. Tak samo łatwo powinny być do wykrycia ataki brute force, polegające na wielokrotnym odgadywaniu haseł po powtarzających się próbach logowania.

2.4. Sposób przetwarzania pakietów a wybór cech

Tradycyjne podejście do trenowania modeli sztucznej inteligencji zakłada ekstrakcję cech z danych wejściowych. Wybór tych cech, zależny od zamierzonego celu, bezpośrednio wpływa na kluczowe kryteria oceny modelu, takie jak wydajność, efektywność i zdolność do generalizacji. W rezultacie problem wyboru cech staje się istotnym zagadnieniem w procesie budowy modeli. Usuwanie cech o

niskiej istotności dla działania modelu czy eliminowanie cech silnie skorelowanych jest standardowym elementem eksploracyjnej analizy danych (EDA), przeprowadzanej przed szkoleniem modelu [12]. W kontekście systemów wykrywania intruzów (IDS) kluczowe pytanie badawcze brzmi: które cechy skonstruowane na podstawie danych pakietowych są najbardziej wartościowe [5] dla zadania wykrywania zagrożeń.

Wybór odpowiednich cech w systemach IDS ma kluczowe znaczenie dla wydajności, szybkości działania oraz zdolności modelu do generalizacji. Trafne zidentyfikowanie istotnych cech pozwala na redukcję wymiarowości danych, co przekłada się na szybsze działanie systemu oraz zmniejszenie zapotrzebowania na zasoby obliczeniowe. Dobrze dobrane cechy zwiększają również efektywność wykrywania, ponieważ eliminują ryzyko wprowadzenia zbędnych informacji, które mogłyby obniżyć precyzję modelu lub prowadzić do przeuczenia. Zdolność do generalizacji wzrasta dzięki zastosowaniu cech pozwalających skutecznie identyfikować wzorce w danych, co czyni model bardziej odpornym na różnorodność i nieznane przypadki. Nieoptymalny dobór cech może jednak skutkować większą podatnością na błędy, co negatywnie wpływa na dokładność systemu oraz wskaźniki takie jak liczba fałszywych alarmów.

Aby sprostać wyzwaniom związanym z wyborem optymalnych cech, które jak wynika z badań [5] często są trudne lub wręcz niemożliwe do jednoznacznego określenia, zaprojektowałem model samodzielnie dokonujący ekstrakcji istotnych cech. Model ten wykorzystuje techniki uczenia głębokiego, gdzie sieci neuronowe w warstwach ukrytych automatycznie identyfikują i przetwarzają kluczowe wzorce w danych. Dzięki temu jest on w stanie dynamicznie dostosowywać swoje reprezentacje do różnych zbiorów danych (representation learning) [8], eliminując konieczność ręcznego doboru cech i minimalizując ryzyko pominięcia istotnych informacji. Takie podejście zwiększa odporność procesu uczenia na błędy związane z niewłaściwym wyborem cech, jednocześnie poprawiając zdolność modelu do generalizacji i adaptacji w nowych, nieznanych scenariuszach.

Model hybrydowy przetwarza dane wejściowe w postaci surowych tablic bajtów (raw byte array) z pliku o formacie .pcap [13], co umożliwia mu dostęp do pełnych informacji zawartych w pakietach sieciowych. Każdy bajt z pakietu jest reprezentowany jako wartość z zakresu od 0 do 255, a następnie mapowany w taki sposób, aby utworzyć jednokanałowy obraz. Dzięki temu dane wejściowe stają się odpowiednie do przetwarzania przez jednowymiarowe konwolucyjne sieci neuronowe (1D-CNN). Taka architektura pozwala modelowi na analizowanie przestrzennych zależności w obrębie pakietu, które są istotne dla identyfikacji wzorców mogących wskazywać na potencjalne zagrożenia. Ujęcie pakietu w formie sekwencji bajtów, przy jednoczesnym zachowaniu informacji o jego strukturze, umożliwia automaty ekstrakcję cech przez warstwy konwolucyjne, co zwiększa precyzję i efektywność wykrywania zagrożeń.

Warto wspomnieć o koncepcjach one-hot encoding i embedding w kontekście porównania z podejściem raw byte array. Techniki takie jak one-hot encoding i embedding różnią się sposobem reprezentacji danych. One-hot Encoding polega na przekształceniu każdej wartości bajta w wektor o wymiarze równym liczbie możliwych wartości (np. 256 dla bajtów), gdzie tylko jeden element przyjmuje wartość 1, a pozostałe są ustawione na 0. Takie podejście znacząco zwiększa wymiarowość danych, co może prowadzić do większego zapotrzebowania na zasoby obliczeniowe oraz komplikować analizę dużych zbiorów danych, takich jak zrzut ruchu sieciowego (jego pakietów). Z kolei embedding redukuje wymiarowość, przypisując każdej wartości bajta wektor o mniejszej liczbie wymiarów, jednak wymaga dodatkowego etapu uczenia w celu wyznaczenia optymalnych reprezentacji. Obie metody generują wielowymiarowe wektory jako wynikowe formaty danych, co może być obciążające dla pamięci, szczególnie w przypadku dużych zestawów danych. Lepszym rozwiązaniem w wielu przypadkach jest bezpośrednie wykorzystanie pojedynczej wartości bajta, zamiast reprezentowania go jako wielowymiarowy wektor.

2.5. Problemy dotyczące danych wejściowych

Aby zapewnić wysoką wydajność modelu, niezbędny jest wysokiej jakości zbiór danych. W uczeniu nadzorowanym odpowiednio przygotowany zbiór danych odgrywa kluczową rolę, ponieważ umożliwia modelowi analizę wielu próbek i identyfikację wzorców. Im większa liczba dostępnych próbek, tym większa szansa na poprawę działania modelu.

Istotnym pytaniem badawczym jest to, jak powinien wyglądać dobry zbiór danych treningowych i jakie cechy powinien posiadać. W kontekście systemów IDS zbiór danych powinien jak najwierniej odwzorowywać rzeczywiste ataki wykorzystywane przez cyberprzestępców w codziennych warunkach sieciowych. Problem polega jednak na tym, że większość dostępnych zbiorów danych nie jest wystarczająco reprezentatywna dla rzeczywistych zagrożeń [4]. Stanowi to istotne wyzwanie dla projektantów systemów IDS oraz specjalistów ds. cyberbezpieczeństwa, którzy muszą bazować na ograniczonych i często nieaktualnych danych.

Dodatkowym utrudnieniem jest sam proces zbierania danych o ruchu sieciowym. Ograniczenia wynikające z regulacji prawnych dotyczących prywatności i ochrony danych osobowych znacząco utrudniają pozyskiwanie próbek z różnych źródeł. Nawet jeśli podmioty byłyby skłonne udostępniać swoje dane, instytucje nadzorujące, takie jak Urząd Ochrony Danych Osobowych, mogą blokować tego typu inicjatywy, zmuszając administratorów do zachowania poufności. RODO nakłada obowiązek minimalizacji zbieranych danych osobowych, co dodatkowo komplikuje proces tworzenia reprezentatywnych zbiorów. Ponieważ ruch sieciowy często zawiera dane pozwalające na identyfikację użytkowników, jest on objęty restrykcyjnymi regulacjami.

Dane pochodzące z wielu źródeł są kluczowe dla zapewnienia reprezentatywności zbioru, ponieważ umożliwiają określenie, czy dany atak rzeczywiście jest powszechny oraz czy można znaleźć wspólne wzorce między różnymi incydentami. Dodatkowo, liczba ataków jest znikoma w porównaniu do normalnego ruchu sieciowego, co zwiększa ryzyko niedostatecznej reprezentacji próbki ataków w zbiorach danych.

Kolejnym wyzwaniem jest ogromna różnorodność ataków. Ponieważ podatności w oprogramowaniu są stopniowo wykrywane i łatane, cyberprzestępcy stale modyfikują swoje techniki, eksplorując nowe luki i dostosowując metody działania. Oznacza to, że struktura nowych ataków może znacząco różnić się od wcześniej znanych zagrożeń. Systemy IDS muszą być w stanie adaptować się do tych zmian. Ataki można podzielić na wiele kategorii, z których każda wymaga odmiennego podejścia do detekcji, co wiąże się z koniecznością dedykowanego szkolenia modelu dla różnych typów zagrożeń.

Ważnym aspektem jest także wybór odpowiednich cech dla zbioru danych. Nowe ataki mogą posiadać unikalne wzorce, które nie były uwzględnione w wcześniejszych zbiorach, co może sprawić, że dotychczasowy zestaw cech stanie się nieaktualny i wymagać będzie rozszerzenia lub modyfikacji. Dodatkowo, różnorodność ataków powoduje, że każda kategoria może wymagać innego zestawu cech do ich precyzyjnego opisania. Jeżeli celem jest stworzenie uniwersalnego systemu IDS, który wykrywa różne rodzaje zagrożeń, konieczność dodawania nowych cech może prowadzić do problemu eksplozji wymiarowości. Może to wymagać ponownego trenowania modelu na bardziej reprezentatywnych danych lub aktualizacji cech opisujących ruch sieciowy.

Warto również zwrócić uwagę na wyzwania związane z wykrywaniem anomalii w ruchu sieciowym. System IDS uznaje za anomalie wszystko, co odbiega od normy wyznaczonej na podstawie zbioru treningowego. Jednak to, co jest uznawane za normę, jest często względne i zależy od specyfiki działania danego urządzenia sieciowego. Na przykład ruch P2P pochodzący od klientów BitTorrent może być uznany za nietypowy dla serwerów SSH, ale nie będzie niczym dziwnym dla witryn oferujących pliki do pobrania. Dodatkowo, użytkownicy mogą wchodzić w interakcje z usługami sieciowymi w sposób niestandardowy, np. w wyniku błędów w oprogramowaniu, co niekoniecznie oznacza zagrożenie. Z drugiej strony, ataki DoS/DDoS czy działania botnetów mogą występować na tyle często, że model przestanie traktować je jako anomalie i nie wykryje zagrożenia. Sama detekcja anomalii nie

jest więc wystarczającym kryterium do klasyfikowania ruchu jako szkodliwego.

2.6. Problem fałszywych pozytywów i negatywów

Problem fałszywych pozytywów jest jednym z najistotniejszych wyzwań w systemach IDS (Intrusion Detection Systems) [6]. W kontekście tej pracy odnosi się do błędnego klasyfikowania pakietów sieciowych jako szkodliwych lub nieszkodliwych w ramach binarnej klasyfikacji. Dla efektywności systemów IDS kluczowe jest utrzymanie niskiego poziomu fałszywych alarmów, które mogą znacząco utrudniać ich działanie. Głównym celem tych systemów jest wykrywanie potencjalnych zagrożeń i przekazywanie ich do dalszej analizy przez odpowiednie osoby. Nadmiar fałszywych alarmów może prowadzić jednak do tzw. "zmęczenia alarmowego," które sprawia, że ostrzeżenia są ignorowane, lub obciąża operatorów koniecznością ręcznego weryfikowania zbyt dużej liczby potencjalnych incydentów. Szczególnym wyzwaniem są systemy bazujące na wykrywaniu anomalii, które cechują się zwiększoną liczbą fałszywych pozytywów. Kluczowe staje się zatem wypracowanie równowagi pomiędzy nadmiernym filtrowaniem, prowadzącym do błędnych klasyfikacji, a ograniczoną czułością, skutkującą przeoczeniem rzeczywistych zagrożeń.

Klasyfikacja binarna, jak sama nazwa wskazuje, opiera się na podziale wyników na pozytywne i negatywne.

Poniższa tabela ilustruje cztery możliwe wyniki w klasyfikacji binarnej:

Tabela 2.1: Macierz Pomyłek (Confusion Matrix) dla Klasyfikacji Binarnej

	Rzeczywista Klasa	
Przewidziana Klasa	**Pozytywna (P)**	**Negatywna (N)**
Pozytywna (P')	**Prawdziwie Pozytywne (TP)**	**Fałszywie Pozytywne (FP)**
Negatywna (N')	**Fałszywie Negatywne (FN)**	**Prawdziwie Negatywne (TN)**

W kontekście systemów detekcji intruzji, poszczególne składowe macierzy pomyłek (ang. confusion matrix) precyzyjnie definiują skuteczność klasyfikacji binarnej:

- Prawdziwie Pozytywne (True Positive – TP): Reprezentuje instancje, w których system poprawnie zidentyfikował faktyczny atak, czyli ruch sieciowy został właściwie sklasyfikowany jako szkodliwy.
- Prawdziwie Negatywne (True Negative – TN): Odnosi się do przypadków, w których system trafnie rozpoznał brak ataku, poprawnie klasyfikując normalny, bezpieczny ruch.
- Fałszywie Pozytywne (False Positive – FP): Oznacza wystąpienie fałszywego alarmu, gdzie normalny ruch sieciowy został błędnie zinterpretowany i sklasyfikowany jako atak. W terminologii statystycznej jest to również określane jako błąd I rodzaju.
- Fałszywie Negatywne (False Negative – FN): Wskazuje na pominięcie ataku, czyli sytuację, w której rzeczywisty atak nie został wykryty i został błędnie sklasyfikowany jako normalny ruch. Ten typ błędu jest znany w literaturze jako błąd II rodzaju.

Fałszywe negatywy w systemach IDS (Intrusion Detection Systems) stanowią równie poważne wyzwanie jak fałszywe pozytywy, gdyż oznaczają sytuacje, w których rzeczywiste zagrożenia nie zostają wykryte i pozostają niezauważone przez system. Takie okazje są szczególnie niebezpieczne, ponieważ nie zgłoszenie potencjalnego ataku może prowadzić do wyrządzenia szkód, szczególnie gdy mowa o zastosowaniu takiego modelu SI lub systemu IDS w środowisku biznesowym. W związku z

tym istotnym wymaganiem staje się ograniczenie liczby fałszywych negatywów. Wykrywanie nowych zagrożeń można rozdzielić na dwa scenariusze: ataki, które są nowymi wariantami wcześniej znanych i wykrywanych przez model oraz zupełnie nowe techniki ataków, nieobecne w zbiorze treningowym, które z dużym prawdopodobieństwem nie zostaną rozpoznane. Model powinien skuteczniej identyfikować wariacje znanych ataków niż zupełnie nowe rodzaje zagrożeń. Opracowany model powinien wykazywać przynajmniej częściową skuteczność w przypadku wariacji znanych ataków. Jeżeli skuteczność modelu okaże się niska to można sprawdzić, czy zwiększy się ona po dodaniu jakiejś warstwy modelu zdolnej wykrywać anomalie jak na przykład autoenkodery albo techniki klastrowania, które mogą zwiększyć czułość modelu na nowe zagrożenia. W przypadku hybrydowego modelu ryzyko fałszywych negatywów może wynikać z ograniczonej zdolności modelu do generalizacji do znajdowania nowych zagrożeń. Ponadto, nieodpowiednia reprezentacja zagrożeń w stosunku do ruchu normalnego w zbiorze treningowym w skutek zbyt niskiej liczby próbek szkodliwego ruchu, może prowadzić do ignorowania anomalii lub ich błędnej klasyfikacji.

2.7. Architektura modelu hybrydowego

W tej sekcji pochyle się nad wewnętrznym działaniem sieci neuronowych oraz warstw sieci stosowanych w moim modelu hybrydowym. Wyjaśnię w jaki sposób model przyjmuje swoje dane treningowe i w jakim formacie udostępnia wyniki swoich przewidywań. Z omawiania teorii zacznę od wyjaśniania reguł, logiki i równań stojących za sztucznymi sieciami neuronowymi, opisze pokrótce proces nauki takiej sieci czyli algorytm propagacji w przód i wstecz. Opiszę różnice pomiędzy różnymi rodzajami sieci neuronowych zintegrowanych jako poszczególne warstwy modelu hybrydowego. Na samym końcu wyjaśnię jakie warstwy po kolei wykorzystałem w moim modelu oraz jakie hiper-parametry z nimi związane ustawiłem.

2.7.1. Sposób wczytywania i otrzymywania danych

Celem modelu jest analiza pakietu w postaci surowej z pliku .pcap (plik ruchu sieciowego). Większość modelów będzie wykorzystywała pliki ruchu sieciowego bezpośrednio, jednakże planowane w przyszłości jest wprowadzenie przetwarzania tych plików na plik csv, w celu dodawania nowych cech (np. wolumetrycznych) oraz skrócenia czasu ładowania zbiorów danych. Wtedy na podstawie pliku pcap byłyby generowane dwa pliki csv.

Pierwszy z nich będzie przedstawiał każdy pakiet w osobnym wierszu wraz z potrzebnymi cechami wydobywanymi z pakietu. Najważniejsza cechą będzie postać surowa (raw) pakietu, czyli tablica bajtów pakietu. Pozostałe cechy będą generowane na podstawie najważniejszych pół nagłówków w pakiecie, dla przykładu adresów IP, portów TCP/UDP, protokołu etc. Cechy dodatkowe przydarzą się do testów modelów uczenia maszynowego, bądź dodatkowej sieci w komitecie modeli. Na podstawie najważniejszej cechy działałby główny model z siecią CNN [1] jako pierwszą.

Drugi plik w niektórych modelach będzie zawierał statystyki wolumetryczne w danym ruchu. Przykładem takich statystyk może być średnia ilość pakietów na sekundę czy średnia ilość bajtów na sekundę. Cechy te mają pozwolić na łatwe zidentyfikowanie ataków DDoS i botnetów, które stanowią jedne z najbardziej popularnych ataków. Pakiet jest uważany za należący do pewnego ruchu, jeżeli pewne warunki są spełnione: pakiety w ruchu zawierają te same wybrane cechy identyfikujące (np. adresy IP, porty, protokół komunikacji), oraz ostatni pakiet był wysłany nie później niż x sekund (długość okna czasowego), gdzie x to zmienna określająca maksymalny czas pomiędzy pakietami, tak aby pakiet był uznawany że należący do danego konkretnego ruchu. Oznacza to że może być w pliku .pcap wiele różnych ruchów z tego samego źródła, jeżeli odstęp pomiędzy ruchami, będzie przewyższał tą zmienną - długość okna. Oznacza to też także że sam ruch może trwać dłużej niż standardowa długość

okna, jeżeli jest regularnie przedłużany. Można także próbować klasyfikować jakość źródła ruchu, na podstawie tego pliku, co może uprościć działanie modelu. Wtedy pakiety z źródła uznanego za niebezpieczne byłyby automatycznie klasyfikowane jako stanowiące zagrożenie, co mogłoby pomóc w przypadku ataków wolumetrycznych.

Dla modelu bazowego będą testowane dwie różne metody podawania danych wejściowych. Główną metodą będzie podawanie danych w postaci surowej, czyli tablicy surowych bajtów jako cech modelu. Innym testowanym podejściem będzie konwersja tej tablicy w dwuwymiarowy kwadratowy obraz oraz zastosowanie techniki one hot encoding.

Dodatkowo też przygotowałem skrypt do generacji syntetycznego ruchu sieciowego, gdyby obecny zbiór danych okazał się niewystarczający. Skrypt ten pozwala na wygenerowanie przykładowego zbioru danych, w którym generowany jest zarówno normalny ruch jak i krótkie gwałtowne fale (bursty) w których te same adresy IP próbują dokonywać ataków DDoS. Pomimo tego że zbiory syntetyczne mogą nie być, aż tak wiarygodne jak zbiory danych zbierane na podstawie faktycznego realnego ruchu sieciowego, mogą się przydać to wytwarzania pewnych wzorców w danych (czyli przykładowych ataków jak AI powinno się nauczyć) lub do dodania brakujących danych w przypadku imbalansu przypadków ataku do normalnego ruchu w zbiorach treningowych i testowych.

Przy wczytywaniu danych treningowych został zaimplementowany mechanizm usuwania nadmiarowych danych z klasy większościowej, w celu wyrównania imbalansu danych. Usuwanie danych można uzasadnić sporymi rozmiarami danych treningowych, które kompensują ewentualnie usunięte pozycje.

2.7.2. Opis działania sieci neuronowych

Sztuczna sieć neuronowa (ang. Artificial Neural Network – ANN) [7] to model obliczeniowy inspirowany strukturą i funkcjonowaniem ludzkiego mózgu. Sieci te składają się z połączonych ze sobą jednostek obliczeniowych - sztucznych neuronów, zorganizowanych w warstwy: warstwę wejściową, jedną lub więcej warstw ukrytych oraz warstwę wyjściową. Każdy neuron w sieci przetwarza dane wejściowe, stosując funkcję aktywacji, a następnie przekazuje wynik do neuronów w następnej warstwie.

Model sztucznego neuronu to informatyczna próba odwzorowania zarówno działania jak i budowy biologicznego neuronu. Przewiduje on obecność jednej lub wielu różnych cech (danych wejściowych) do jednego wyjścia, próbując analogicznie odwzorować połączenie wielu dendrytów do jednego aksonu. Dane wejściowe podawane są w formie wektora gdzie każdy jego komponent odpowiada danej cesze każdego obiektu wejściowego, identyfikowanego przez swoje cechy.

Obliczanie wartości (czyli operacja propagacji w przód) pojedynczego sztucznego neuronu na podstawie przekazanych n danych wejściowych można opisać wzorem [14]:

$$y = f(b + \sum_{i=1}^{n} x_i \cdot w_i)$$

gdzie:

y - obliczona wartość danej wyjściowej

f - wybrana funkcja aktywacyjna, która na podstawie swojej wartości decyduje o "zapaleniu się" neuronu

b lub w_0 - wartość wyrazu przesunięcia (bias)

x_i - wartość i-tej danej wejściowej z n wszystkich możliwych wejść

w_i - obliczona, w wyniku procesu uczenia się, wartość wagi dla konkretnej i-tej cechy (danych wejściowych) z wszystkich n możliwych

W praktycznych realizacjach sieci neuronowych liczba neuronów przekracza pojedynczą jednostkę, a poszczególne neurony organizowane są w warstwy ze zmienna ich liczbą. Liczba neuronów w każdej warstwie stanowi istotny parametr modelu. Zmiana wielkości warstw może wpływać na efektywność działania sieci: zarówno zmniejszanie, jak i zwiększanie liczby neuronów pomiędzy warstwami może prowadzić do poprawy wydajności, zależnie od konkretnej architektury i zadania. W związku z tym niezbędne jest przeprowadzenie eksperymentów w celu określenia optymalnego rozmieszczenia neuronów.

Obliczanie wartości dla prostej sieci neuronowej o dwóch warstwach ukrytych (z jednym neuronem na pierwszej warstwie i jednym neuronem ukrytym) opisuje wzór [7]:

$$y = (g^2(g^1(xW^1 + b^1))W^2 + b^2)W^3$$

gdzie:

y - obliczona wartość danej wyjściowej

g_i - wybrana i-ta funkcja aktywacyjna, która na podstawie swojej wartości decyduje o "zapaleniu się" i-tego neuronu

b_i - wartość wyrazu przesunięcia (bias) i-tego neuronu

W_i - wartość wagi dla i-tego neuronu, obliczona w procesie uczenia się sieci

x - wartość danej wejściowej podawanej do sieci

Można ten wzór uogólnić od dowolnej (większej od 2) liczby neuronów i warstw [14]:

$$a_j^{(l)} = f(\sum_{i=1}^{n} w_{ij}^{(l)} \cdot a_i^{(l-1)} + b_j^{(l)})$$

gdzie:

$a_j^{(l)}$ - wektor wyjść j-tego neuronu warstwy l,

f - funkcja aktywacyjna neuronu j-tego,

$w_{ij}^{(l)}$ - macierz wag połączenia między neuronem j danej warstwy l a i-tym neuronem warstwy poprzedniej $l-1$,

$a_i^{(l-1)}$ - funkcja aktywacji i-tego neuronu warstwy poprzedniej $l-1$,

$b_j^{(l)}$ - wektor wyrazu przesunięcia (ang. bias) j-tego neuronu warstwy l,

Sieci neuronowe oprócz samego mechanizmu obliczania wartości na podstawie wejść, czyli w praktyce akcji przewidywania, muszą posiadać mechanizm dopasowywania wag do oczekiwanych wejść (w przypadku uczenia nadzorowanego). Odpowiedzialny za to będzie proces propagacji wstecznej (lub "w tył") [15], w którym na podstawie popełnionych przez sieć błędów przewidywania (ocenianych wg wybranej metryki celności), wagi (i wartości bias) poszczególnych neuronów będą stopniowo dopasowywane w kolejnych epokach, tak aby te błędy minimalizować.

Proces uczenia się sieci odbywa się za pomocą ciągłego powtarzania tych czterech kroków co epokę:

- Obliczenie aktywacji sieci poprzez propagację w przód (który został wyżej omówiony),
- Obliczenie wartości funkcji błędu na podstawie przyjętej metryki poprawności (np. entropii krzyżowej lub błędu średnio-kwadratowego),
- Wyznaczenie gradientów funkcji straty względem każdego neuronu (pochodne cząstkowe),
- Aktualizacja wag zgodnie z obliczonymi gradientami dla każdego neuronu, ukierunkowana na minimalizację błędu.

Propagacja wsteczna wymaga zdefiniowania funkcji straty, którą sieć neuronowa (konkretnie optymalizator) ma minimalizować podczas procesu uczenia. Najczęściej używane funkcje straty to MSE(ang. mean square error - błąd średniokwadratowy) w przypadku regresji, log loss czy cross-entropy loss. Funkcja strat cross-entropy, z której korzysta mój model opisana jest wzorem [14]:

$$E = L_{cross-entropy}(\hat{y}, y) = -\sum_{i}^{n} y_i log(\hat{y}_i) + (1 - y_i) log(1 - \hat{y}_i)$$

gdzie: $\hat{y}$ - wartość wyjściowa przewidziana przez sieć neuronową y - faktyczna wartość wyjściowa (etykieta) z zbioru danych n - ilość próbek danych

Różnice wag i przesunięć neuronów, otrzymane w wyniku propagacji wstecznej, otrzymuje się wzorem: [14]:

$$\delta w_{i,j}^{(l)} = -\eta \frac{\partial E}{\partial w_{i,j}^{(l)}}$$

$$\delta b_{j}^{(l)} = \eta \frac{\partial E}{\partial b_{j}^{(l)}}$$

gdzie: $w_{i,j}^{(l)}$ - waga połączenia neuronu j w warstwie l z neuronem i poprzedniej warstwy $(l-1)$ $b_j^{(l)}$ - wyraz przesunięcia (bias) neuronu j w warstwie l η - zmienna oznaczająca prędkość uczenia sieci (ang. learning rate) E - wybrana funkcja straty

W wyniku procesu propagacji wstecznej, dzięki optymalizatorom zarówno waga neuronu jak i wartość przesunięcia (bias) jest dopasowywana zgodnie z wzorami [14]:

$$w_{i,j}^{(l)} = w_{i,j}^{(l)} + \delta w_{i,j}^{(l)}$$

$$b_{j}^{(l)} = b_{j}^{(l)} + \delta b_{j}^{(l)}$$

gdzie: $w_{i,j}^{(l)}$ - waga połączenia neuronu j w warstwie l z neuronem i poprzedniej warstwy $(l-1)$ $b_j^{(l)}$ - wyraz przesunięcia (bias) neuronu j w warstwie l

Warto też wspomnieć że istnieją różne optymalizatory funkcji straty takie jak: SGD (ang. Stochastic Gradient Descent), RMSProp, AdaGrad czy często używany Adam [19]. Wybrany przeze mnie optymalizator Adam charakteryzuje się niskim zapotrzebowaniem pamięciowym i jest wydajny obliczeniowo [19], co sprawia, że idealnie sprawi się w moim modelu hybrydowym. Różnica pomiędzy czystym algorytmem SGD, a innymi wyżej wymienionymi optymalizatorami, polega na sprawieniu, aby parametr jakim jest prędkość uczenia był dynamicznie zmieniająca się zmienną. Algorytm Adam, wykorzystuje dwa dodatkowe (względem prędkości uczenia nazywanej parametrem alfa) parametry β_1 oraz β_2, które odpowiadają za zmniejszanie (ang. decay rate) parametru szybkości uczenia.

2.7.3. Opis działania sieci konwolucyjnych

Sieci konwolucyjne (ang. Convolution Neural Network - CNN) [18] to rodzaj głębokiej sieci neuronowej, która jak sama nazwa wskazuje wykorzystuje operacje konwolucji (po polsku splotu) do uczenia się. Sieci te zostały przemyślane z myślą o przetwarzaniu obrazów, jednakże znajdują także zastosowanie w wielu innych dziedzinach, między innymi w zastosowaniach przetwarzania języka naturalnego (NLP) takich jak klasyfikacja dokumentów, klasyfikacji krótkiego tekstu czy analiza sentymentu[7]. Specyfiką sieci konwolucyjnych jest to, że potrafią analizować strukturę przestrzenną podanego wejścia, stąd świetnie nadają się do przetwarzania obrazów.

Kluczowym dla tej sieci jest operator splotu. W znaczeniu ogólnym splot funkcji f(x) i g(x) opisuje się wzorem $(f * g)(x) = \int_t f(t-x)g(x)dt$. W przypadku sieci CNN mówimy raczej o splocie dyskretnym, który jest wykonywany na macierzach (realizowanych jako tablice) na podstawie obrazu wejściowego. Splot dyskretny dla funkcji f(x) i g(x) opisuje wzór $(f * g)(x) = \sum_t f(x-t) \cdot g(x)$

W sieciach CNN filtr (lub jądro) przesuwa się po obrazie wejściowym, działając niczym okno, które na każdym kroku wycina fragment obrazu. Dla każdego przesunięcia obliczany jest iloczyn skalarny między wartościami pikseli wyciętego fragmentu a wagami filtra, a następnie dodawany jest wyraz przesunięcia (bias). Wynikiem tej operacji jest mapa cech, która przedstawia wykryte wzorce w danych. Równanie na wartość cechy dla danej pozycji warstwy stosowane w CNN ma postać [1]:

$$z^l_{i,j,k} = (w^l_k)^T \cdot x^l_{i,j} + b^l_k$$

gdzie:

$z^l_{i,j,k}$ - wartość cechy na pozycji (i, j) w k-tej mapie cech warstwy l
w^l_k - wartość wag (filtrów) odpowiadający k-tej mapie cech warstwy l
$x^l_{i,j}$ - fragment danych wejściowych dla pozycji (i, j) dla warstwy l
b^l_k - wyraz przesunięcia (bias) dla k-tego filtra dla warstwy l

Równanie te opisuje sposób tworzenia map cech dla danej warstwy l. Jednak w typowej sieci warstw konwolucji będzie więcej niż jedna po to, żeby polepszyć działanie sieci. Każda kolejna warstwa, która będzie dokładana, ma za zadanie zwiększyć dokładność sieci. Dla przykładu słynna sieć CNN - LeNet-5 [16] posiadała tytułowe 5 różnych warstw, z czego dwie z nich to były warstwy konwolucji.

Oprócz tego warto wspomnieć, że wartość cechy na danej pozycji w mapie cech zostaje podawana do wybranej funkcji aktywacyjnej. Jest wiele funkcji aktywacyjnych m.in sigmoid ($\frac{1}{1+e^{-x}}$), tanh (tangens hiperboliczny), ReLU (Rectified Linear Unit). Dla dobrego działania sieci wielowarstwowych kluczowe jest aby wybierać funkcje aktywacyjne, które są nieliniowe, jak te które wyżej wymieniłem. Bardzo przydatna okazuje się funkcja aktywacyjna ReLU, której nagminnie będę stosował w modelach. Finalne równanie z uwzględnieniem funkcji aktywacyjnych wyglądało by tak:

$$a^l_{i,j,k} = a(z^l_{i,j,k}) = a((w^l_k)^T \cdot x^l_{i,j} + b^l_k)$$

W typowej sieci CNN będą także wykorzystywane inne warstwy niż sama konwolucyjna, która stanowi jej rdzeń, główną część. Pomiędzy warstwami konwolucji, bardzo często będą stać różnego rodzaju warstwy typu pooling.

Mają one za zadanie zmniejszać wymiary obrazka źródłowego/mapy cech przy zachowaniu większości informacji. Znane są różne warianty warstw pooling, choć najczęściej mówi się albo o max pooling, czyli biorąca maksymalną wartość z filtra oraz o average pooling, która analogicznie będzie otrzymywała średnią wartość z filtra.

Trzecią najczęściej wykorzystywaną warstwą w sieciach CNN poza warstwą konwolucji i pooling będzie warstwa w pełni połączona lub gęsta (fully-connected, dense). Warstwa ta jest często wykorzystywana, aby przekonwertować mapę cech z poprzedniej warstw konwolucji do postaci liczby, czego wymaga zadanie klasyfikacji. Bardzo często korzysta się z warstw dense dopiero na końcu modelu i to w większej ilości warstw dense (niż jedna). Przykładem ilustrującym to może być np. wcześniej wspominana sieć LeNet-5, która posiada na końcu 3 warstwy dense, każda to z coraz to mniejszą ilością neuronów. Ostatnia warstwa dense, jeżeli jest więcej niż jedna, musi zawierać tyle neuronów ile jest klas przewidywanych przez model. Dodanie dodatkowych warstw wiąże się z dodaniem stanu ukrytego do sieci, który może potencjalnie pozwolić na lepsze lub gorsze wyniki, w zależności od wybranego zadania, zastosowania sieci. Warstwa dense realizuje normalną sieć neuronowa feed-forward, omawianą wcześniej.

Wraz z postępem sieci CNN zaproponowano nowe usprawnienia do procesu konwolucji [1]. Operacja *dilated convolution* (splot rozszerzony) stanowi rozszerzenie klasycznej konwolucji poprzez wprowadzenie dodatkowych odstępów pomiędzy elementami filtra. Odstępy te są kontrolowane przez parametr *dilation rate*, który definiuje liczbę „dziur" (pominiętych elementów wejścia) między sąsiednimi wartościami filtra. Zaletą tej techniki jest możliwość zwiększenia pola widzenia bez obniżania rozdzielczości mapy cech. Dilated convolution znalazła szerokie zastosowanie m.in. w segmentacji semantycznej obrazów oraz w analizie sygnałów sekwencyjnych.

Drugim interesującym rozszerzeniem klasycznej konwolucji jest *tiled convolution*, czyli splot kafelkowy. W standardowych architekturach CNN stosuje się pełne dzielenie wag (ang. *weight sharing*), co oznacza, że ten sam filtr jest stosowany w całym obszarze wejścia. Tiled convolution łamie tę zasadę, wprowadzając parametr *tile size* – określający, co ile pozycji filtr może przyjąć inny zestaw wag. W praktyce oznacza to, że zamiast jednego filtra, w sieci znajduje się zestaw filtrów, z których każdy aktywowany jest okresowo w przestrzeni wejściowej. Przykładowo, dla *tile size* $k = 2$, jeden zestaw wag będzie używany co dwa piksele. Takie podejście zwiększa pojemność modelu i pozwala sieci uczyć się różnych inwariancji (niezmienności pod wpływem przesunięć i rotacji) bez konieczności stosowania wielu filtrów jednocześnie. Rozwiązanie to może być szczególnie przydatne w zadaniach klasyfikacji obrazów z powtarzalnymi, lokalnie zmiennymi wzorcami, w których klasyczny mechanizm weight sharing może być zbyt ograniczający.

Warto zaznaczyć, że sieci konwolucyjne (CNN) nie są jedynym podejściem do analizy danych o strukturze przestrzennej lub sekwencyjnej. Coraz większą popularność zyskują transformery [9], pierwotnie zaprojektowane do przetwarzania języka naturalnego, które dzięki mechanizmowi uwagi (attention) potrafią efektywnie modelować długozasięgowe zależności w danych. Alternatywnym podejściem są również grafowe sieci neuronowe (Graph Neural Networks, GNN) [3], które umożliwiają operowanie na danych o nieregularnej strukturze grafowej, takich jak topologie sieci komputerowych.

Biblioteka TensorFlow (moduł `keras.layers`) udostępnia warstwy `Conv1D` i `Conv2D`, które można wykorzystać do realizacji zarówno standardowych, jak i rozszerzonych wariantów konwolucji. Kluczowe parametry tych warstw opisano poniżej:

- `filters` – liczba filtrów (kanałów wyjściowych), decydująca o liczbie map cech w wyjściu.
- `kernel_size` – rozmiar jądra konwolucyjnego
- `strides` – krok przesunięcia filtra
- `padding` – sposób uzupełniania krawędzi wejścia: `"valid"` (rozmiary wejścia się zmniejszają) lub `"same"` (zachowanie rozmiaru wejścia).
- `dilation_rate` – parametr kontrolujący dylatacje (realizujący wcześniej opisany dilated convolution)
- `activation` – funkcja aktywacji, np. `relu`, `tanh`, `sigmoid`.
- `use_bias` – czy dodać wyraz przesunięcia (bias).
- `kernel_initializer`, `bias_initializer` – sposób inicjalizacji wag i biasów.
- `kernel_regularizer`, `bias_regularizer` – zastosowanie technik regularyzujących, np. L2.
- `input_shape` – kształt wejścia wymagany w pierwszej warstwie modelu.

2.7.4. Opis działania sieci rekurencyjnych

Rekurencyjne sieci neuronowe (ang. Recurrent Neural Network - RNN) [17] to kategoria sieci neuronowych, która wykorzystuje rekurencyjne połączenia do uczenia się danych. Sieci te są przemyślane do wykonywania zadań takich: jak sekwencje czasowe, język naturalny czy rozpoznawanie mowy, co oznacza że świetnie przetwarzają dane sekwencyjne.

Sieci RNN mają wbudowaną "pamięć" przeszłych wartości, co pozwala im uwzględniać kontekst wcześniejszych danych podczas przetwarzania aktualnych danych, co usprawnia proces uczenia. W przeciwieństwie do normalnych (feedforward) sieci neuronowych, neurony formują cykle, w taki sposób że warstwy neuronów mają swoje wyjście które staję się własnym wejściem. Dzięki czemu sieć jest w stanie pamiętać swój poprzedni stan i przewidywać nowe wartości na podstawie poprzedniego, co daje mu wcześniej wspomniany "kontekst".

Algorytm propagacji wstecznej różni się pomiędzy normalnymi (w przód) a rekurencyjnymi sieciami neuronowymi. Sieci RNN wykorzystują algotytm BPTT (Backpropagation through time) [14], który różni się od klasycznej propagacji wstecznej obecnością kroków czasowych, wynikających z rekurencyjnych połączeń. Konieczne jest uwzględnienie czasu jako zmiennej (stąd kroki czasowe) w gradiencie zmiany wartości przewidywanej od wag neuronów, który jest konieczny do nauczania sieci.

Pomimo dobrej skuteczności RNN w wielu zastosowaniach, ten typ sieci też cierpi na swoje problemy jak wybuchający czy zanikający gradient. W przypadku zanikającego gradientu, staje się on tak niski, że sieć przestaje się uczyć. Zbyt duży zaś prowadzi do spadku celności i nadmiernego zwiększania wag. Zjawiska te są odpowiedzialne za pogarszanie celności sieci, jest to więc problem, który trzeba uwzględnić przy konstrukcji modelów, tak aby minimalizować straty celności.

Rozwiązaniem problemu zanikającego/wybuchającego gradientu stało się wykorzystywanie sieci/warstw LSTM (ang. Long Short-Term Memory) [2], które są podkategorią sieci rekurencyjnych. Od klasycznych sieci rekurencyjnych, sieci LSTM różnią się swoją budową, która pozwala im na zapominanie obecnego stanu. Budowa komórki LSTM zawiera następujące elementy: stan obecny, bramki uaktualnienia oraz bramkę zapominania, która okazuje się kluczowym elementem w tej sieci.

2.7.5. Opis warstw modelu i ich parametrów

Główny model SI badany w tej pracy łączy w sobie technologie CNN+LSTM oraz przyjmowanie informacji w formacie tablicy surowych bajtów. Dane wejściowe mają postać jednowymiarowych tablic bajtów o stałej długości (150 bajtów), znormalizowanych przez podzielenie przez 256.

Model rozpoczyna się od dwóch warstw konwolucyjnych, których zadaniem jest ekstrakcja lokalnych wzorców w sekwencjach bajtowych. W modelach zastosowano warstwy konwolucyjne (Conv1D i Conv2D) oraz warstwy spłaszczania przestrzeni (MaxPooling1D i MaxPooling2D), które odgrywają kluczową rolę w ekstrakcji cech z danych wejściowych. W przypadku Conv2D, parametry takie jak liczba filtrów (np. 64 lub 128) oraz rozmiar jądra (np. 3 x 3) określają, ile lokalnych wzorców model będzie próbował wykryć i jak duży fragment danych będzie analizowany w jednym kroku. Funkcja aktywacji (sigmoid, tanh lub relu) decyduje o nieliniowym przekształceniu wykrytych cech. Z kolei warstwa MaxPooling, z parametrem pool size (często równym 2 lub 3), odpowiada za redukcję wymiarów danych poprzez wybieranie maksymalnych wartości z lokalnych okien, co pozwala zachować najistotniejsze informacje przy jednoczesnym ograniczeniu liczby parametrów i przyspieszeniu uczenia.

W modelu zastosowano również warstwę LSTM (Long Short-Term Memory), która służy do przetwarzania sekwencji i wychwytywania zależności czasowych lub pozycyjnych pomiędzy elementami wejściowymi będącymi bajtami pakietu sieciowego. W modelu wykorzystano wariant Bidirectional, który analizuje dane w obu kierunkach (od początku do końca i odwrotnie), co zwiększa zdolność sieci do rozpoznawania wzorców niezależnie od ich położenia w sekwencji. Najważniejszym parametrem

okazuje się ilość jednostek (np. 32 czy 64) oraz to czy model zwraca sekwencje, co jest potrzebne w przypadku podawania wyniku dalej do warstwy Dense lub kolejnej warstwy LSTM.

Z kolei warstwa Flatten pełni funkcję pomocniczą, konwertując dane wielowymiarowe (np. wynik działania warstw konwolucyjnych lub pooling) do postaci jednowymiarowego wektora. Taka forma danych jest niezbędna, aby można je było dalej przetwarzać przez w pełni połączone warstwy (Dense) lub w niektórych przypadkach przekształcić na format wejściowy dla LSTM. Warstwa Flatten nie posiada parametrów do trenowania – jej rola jest czysto strukturalna, ale kluczowa w kontekście przejścia między różnymi typami warstw w modelu.

Część klasyfikacyjna modelu składa się z dwóch warstw gęstych (Dense). Pierwsza zawiera 32,64 lub 128 neurony z funkcją aktywacji ReLU i regularizacją L2 na poziomie 0,001, druga zaś zawsze 4 neurony z funkcją aktywacyjną tanh. Na końcu znajduje się warstwa wyjściowa z jednym neuronem aktywowanym funkcją sigmoid, co pozwala na binarną klasyfikację pakietu jako szkodliwego lub nieszkodliwego. W procesie przewidywania stosowany jest próg decyzyjny równy 0,5 do klasyfikowania pakietu jako zagrożenia.

Model kompilowany jest z użyciem optymalizatora Adam o współczynniku uczenia 0,0002. Jako funkcję kosztu wykorzystano binary cross-entropy. Dodatkowo w kodzie zaimplementowano opcję użycia funkcji straty typu focal loss, jednak w podstawowym eksperymencie jej nie aktywowano. Model trenowany jest z uwzględnieniem wag klas, co kompensuje nierównomierny rozkład danych. Trening odbywa się z parametrami batch size 64 wraz z użyciem dodatkowych funkcji callback implementujących mechanizm early stopping z tolerancją 3 epok oraz redukcji prędkości uczenia (learning rate) przy braku poprawy straty walidacyjnej (val loss) po 2 epokach z parametrami minimalnej prędkości uczenia 0,0001 oraz stopnia zmniejszania 0,1.

Badane także są dwa inne warianty głównego modelu - wariant pomijający warstwę LSTM odpowiedzialna za dodatkowe rozpoznawanie wzorców czasowych oraz wariant który wykorzystuje obie te warstwy lecz przyjmuje dane zakodowane w postaci 2D z one-hot encoding.

Pierwszy wariant (bez LSTM) nie praktycznie nie zmieniał parametrów modelu, poza ilością neuronów w pierwszej warstwie dense, gdzie ich ilość zmniejszyła się o połowę (czyli do 32).

Drugi wariant przyjmujący dane za pomocą 2D one-hot encoding różni się już bardziej znacząco od głównego modelu. Model rozpoczyna się od warstwy Embedding, która mapuje każdy z 256 możliwych bajtów wejściowych na 16-wymiarowy wektor cech oraz ma pozmieniane domyślne parametry względem głównego modelu.

Rozdział 3

Implementacja modelów i porównanie

3.1. Opis porównywanych modeli i parametrów

W ramach eksperymentu porównawczego przetestowanych zostało siedem modeli sztucznej inteligencji, różniących się architekturą oraz sposobem reprezentacji danych wejściowych. Celem tego podejścia jest analiza wpływu zastosowanych technik SI na jakość detekcji oraz ocena skuteczności poszczególnych podejść w zadaniu klasyfikacji pakietów sieciowych. Wszystkie modele zostały zaimplementowane w języku Python z wykorzystaniem popularnych bibliotek takich jak `TensorFlow`, `NumPy` oraz `scikit-learn`, umożliwiających szybkie prototypowanie i trenowanie modeli uczenia maszynowego.

Model konwolucyjny (CNN) Model oparty wyłącznie na warstwach konwolucyjnych (`Conv1D`) został zaprojektowany do bezpośredniej analizy sekwencji bajtów pakietów sieciowych. Dzięki zastosowaniu filtrów konwolucyjnych model jest w stanie wykrywać lokalne wzorce w danych wejściowych, co umożliwia identyfikację struktur charakterystycznych dla szkodliwego ruchu. Dane wejściowe mają postać jednowymiarowego wektora bajtów (`raw byte array`), gdzie każdy bajt jest zakodowany jako wartość z zakresu $[0, 255]$. Architektura ta charakteryzuje się stosunkowo niską złożonością obliczeniową i krótkim czasem uczenia. Dokładne wartości parametrów znajdują się w tabeli poniżej 3.1.

Model hybrydowy CNN+LSTM (raw byte array) Głównym modelem analizowanym w pracy jest hybrydowa sieć neuronowa łącząca warstwy konwolucyjne (CNN) z warstwami rekurencyjnymi typu LSTM (Long Short-Term Memory). Model ten przyjmuje dane w postaci surowych bajtów i wykorzystuje warstwę konwolucyjną do ekstrakcji lokalnych cech przestrzennych, a następnie warstwę LSTM do uchwycenia zależności czasowych i sekwencyjnych pomiędzy pakietami. Takie połączenie umożliwia efektywne modelowanie zarówno lokalnych, jak i długozasięgowych wzorców w ruchu sieciowym. Model ten stanowi główny obiekt badań eksperymentalnych. Dokładne wartości parametrów znajdują się w tabeli poniżej 3.1.

Model hybrydowy CNN+LSTM (OHE 2D) W tej wersji hybrydowego modelu zastosowano jednoczesne wykorzystanie kodowania one-hot (OHE) w postaci macierzy 2D oraz architektury konwolucyjno-rekurencyjnej. Każdy bajt wejściowy jest zakodowany jako wektor o długości 256, a cała sekwencja przyjmuje formę macierzy. Taka reprezentacja, choć bogata w informacje, znacząco zwiększa wymiarowość danych i tym samym obciążenie obliczeniowe. Model ten pozwala ocenić, czy bardziej rozbudowana reprezentacja wejścia wpływa na skuteczność klasyfikacji w porównaniu do podejścia z surowym wejściem. Dokładne wartości parametrów znajdują się w tabeli poniżej 3.1.

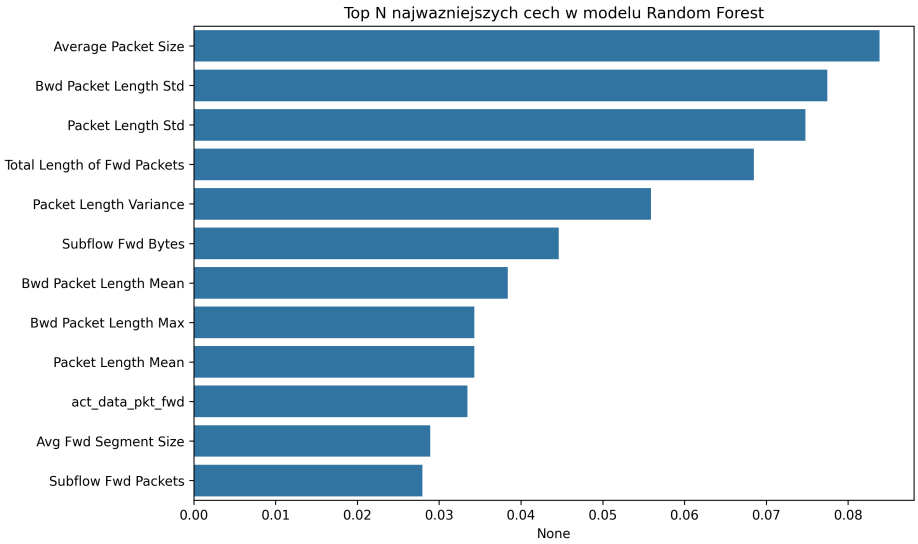
Model oparty na embeddingu, transformatorze i RNN Kolejny model korzysta z podejścia zorientowanego na reprezentację semantyczną, w którym dane wejściowe są przekształcane za pomocą warstwy embedding. Następnie przetwarzane są przez mechanizm uwagi (self-attention) w architekturze typu Transformer znanej z zastosowań NLP, a końcowa sekwencja poddawana jest analizie przez warstwę RNN. Dzięki temu możliwe jest uchwycenie zarówno globalnych zależności w danych, jak i lokalnych wzorców. Dokładne wartość parametrów znajdują się w liście poniżej.

- **Długość sekwencji (PADDING_LENGTH)** – 150 bajtów
- **Zakres bajtów (vocab_size)** – 256 (wszystkie możliwe wartości bajtów)
- **Embedding bajtów** – `Embedding(256, 64)` – każdy bajt kodowany jako 64-wymiarowy wektor
- **Embedding pozycji (positional encoding)** – `Embedding(150, 64)` – unikalny embedding dla każdej pozycji w sekwencji
- **Liczba głowic uwagi (num_heads)** – 4
- **Rozmiar klucza (key_dim)** – 64
- **Mechanizm uwagi (MultiHeadAttention)** – `MultiHeadAttention(num_heads=4, key_dim=64)`
- **Dropout (po attention i FF)** – 0,1
- **Layer Normalization** – Tak (po bloku uwagi i po bloku feed-forward)
- **Feed-forward block (FF_DIM)** – `Dense(128, ReLU)` → `Dense(64)`
- **Pooling** – `GlobalAveragePooling1D()`
- **Reshape do sekwencji (dla LSTM)** – `Reshape((1, 64))` – przekształcenie przed BiLSTM
- **Warstwa rekurencyjna** – `Bidirectional(LSTM(64))`
- **Dropout (przed klasyfikacją)** – 0,3
- **Wyjście** – `Dense(1, activation='sigmoid')` – klasyfikacja binarna
- **Funkcja straty** – `binary_crossentropy`
- **Optymalizator** – `Adam(learning_rate=1e-5)`
- **Batch size** – 64
- **EarlyStopping** – `patience=3`, `restore_best_weights=True`
- **ReduceLROnPlateau** – `patience=2`, `factor=0.1`, `min_lr=1e-5`

Model oparty wyłącznie na warstwach Dense (DNN) Model głębokiej sieci neuronowej (DNN) opiera się wyłącznie na warstwach gęstych (`Dense`), bez wbudowanego mechanizmu samodzielnej ekstrakcji cech. W przeciwieństwie do modeli CNN i embeddingowych, które uczą się reprezentacji cech automatycznie, DNN operuje bezpośrednio na wcześniej zdefiniowanych cechach ze zbioru danych. Model ten pozwala porównać efektywność tradycyjnych metod głębokiego uczenia z nowoczesnym podejściem representation learning.

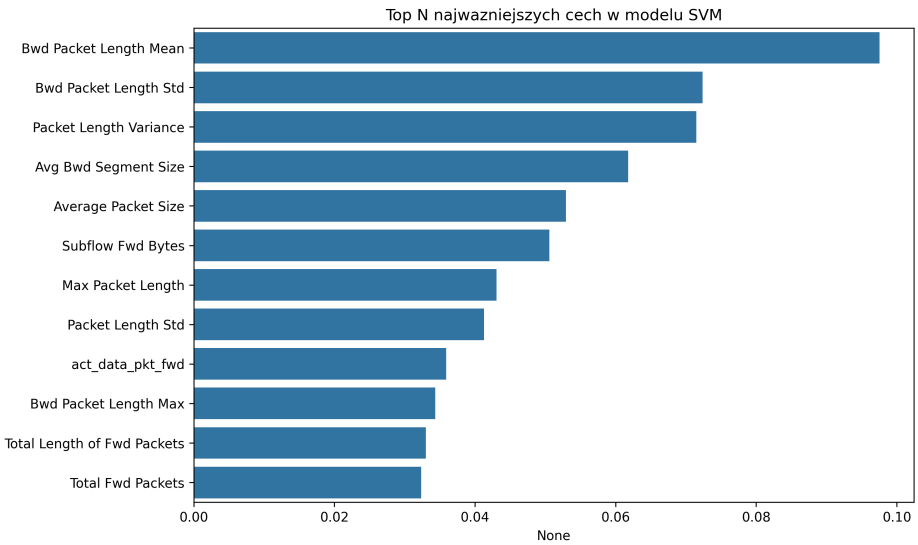
- **Liczba cech wejściowych (input_dim)** – 12 (TOP_FEATURES)
- **Warstwa wejściowa** – `Dense(64)`, funkcja aktywacji `ReLU`
- **Dropout po warstwie 1** – 0,3
- **Warstwa ukryta 2** – `Dense(32)`, funkcja aktywacji `ReLU`
- **Dropout po warstwie 2** – 0,3
- **Warstwa ukryta 3** – `Dense(16)`, funkcja aktywacji `ReLU`
- **Warstwa wyjściowa** – `Dense(1)`, funkcja aktywacji `sigmoid` (klasyfikacja binarna)
- **Funkcja straty** – `binary_crossentropy`
- **Optymalizator** – `Adam(learning_rate=0.0001)`
- **Batch size** – 32
- **Liczba epok** – 20
- **Podział trening/walidacja** – `validation_split=0.2` (czyli 20% danych treningowych jako walidacyjne)
- **EarlyStopping** – `monitor='val_loss'`, `patience=5`, `restore_best_weights=True`
- **ReduceLROnPlateau** – `monitor='val_loss'`, `factor=0.1`, `patience=3`, `min_lr=1e-5`

Model klasyczny: las losowy (Random Forest) Random Forest to klasyczny algorytm uczenia zespołowego (ensemble learning), który wykorzystuje $n = 100$ drzew decyzyjnych w celu poprawy dokładności klasyfikacji. Model ten operuje na wcześniej przygotowanych cechach i nie wykonuje samodzielnej ekstrakcji reprezentacji. Wybierane jest pierwsze dwanaście najważniejszych,najbardziej znaczących cech, wydobywanych za pomocą klasyfikatora Random Forest. Stanowi on popularny punkt odniesienia w wielu zastosowaniach klasyfikacyjnych, charakteryzując się wysoką dokładnością i odpornością na przeuczenie. Dwanaście najważniejszych cech opisuje ilustracja 3.1.



Rysunek 3.1: Wykaz 12 najważniejszych cech modelu RF

Model klasyczny: maszyna wektorów nośnych (SVM) Support Vector Machine (SVM) to kolejny klasyczny model stosowany w zadaniach klasyfikacyjnych. Jego działanie polega na wyznaczeniu optymalnej hiperpłaszczyzny separującej klasy w przestrzeni cech. Model ten jest szczególnie skuteczny w przypadku dobrze zrównoważonych i niskowymiarowych zbiorów danych, lecz może być wrażliwy na problemy z nierównomierną reprezentacją klas, co często występuje w zastosowaniach IDS. Klasyfikatorowi wybrano jądro RBF, z parametrem regularyzacji $C = 1.0$ operującym na 60000 pakietach oraz na 12 najważniejszych cechach uzyskanych 3.2.



Rysunek 3.2: Wykaz 12 najważniejszych cech modelu SVM

Dla wygody wszystkie parametry modelu hybrydowego i jego wariantów zostały zebrane w tabelę 3.1:

Tabela 3.1: Porównanie parametrów modelu hybrydowego wraz z jego wariantami

Model	**CNN**	**CNN + LSTM (raw bytes array)**	**CNN + LSTM (OHE)**
Wejście	Normalizowane bajty (float/256)	Normalizowane bajty (float/256)	One-hot encoded bajty
Długość sekwencji (PADDING_LENGTH)	150	150	150
Embedding	Nie	Nie	Nie
Warstwy konwolucyjne	Conv1D: 64, 32	Conv1D: 64, 32	Conv2D: 128, 64
Aktywacje w CNN	2x sigmoid	2x sigmoid	sigmoid, tanh
Pooling	MaxPooling1D (2)	MaxPooling1D (2)	MaxPooling2D (2×2)
Dropout	0.5 (po Dense)	0.3 (po CNN i Dense)	0.3 (po CNN i Dense)
LSTM	Brak	Bidirectional LSTM: 64 + 32	Bidirectional LSTM: 64 + 32
Warstwy Dense	64 + 4	32 + 4	128 + 64
Regularyzacja L2	Tak ($\lambda = 0.001$)	Tak ($\lambda = 0.001$)	Tak ($\lambda = 0.001$)
Optymalizator	Adam ($lr = 0.001$)	Adam ($lr = 0.0002$)	Adam ($lr = 0.0002$)
Batch size	64	64	64

3.2. Opis programów realizujących modele SI

Aby przeprowadzić eksperymenty porównawcze modeli sztucznej inteligencji, przygotowano spójną strukturę katalogów oraz zestaw skryptów umożliwiających automatyzację procesu testowania. Główna organizacja folderów obejmuje trzy podstawowe foldery, z czego dwa z nich odpowiadają za dane treningowe oraz zestaw skryptów w python i bash.

- `CIC IDS 2017/` – folder zawierający przetworzony zbiór danych pochodzący z otwartego zestawu danych *CIC IDS 2017*, będącego jednym z popularnych benchmarków do badań nad systemami wykrywania włamań.
- `Dataset CTU-13/` – katalog zawierający dane z zestawu *CTU-13*, skupiającego się na analizie ruchu sieciowego generowanego przez botnety.
- `TestCases/` – folder zawierający wszystkie przypadki testowe w postaci skryptów Pythona (`.py`), z których każdy reprezentuje inny model sztucznej inteligencji. Każdy z plików zawiera kompletną definicję architektury modelu, konfigurację hiperparametrów oraz kod odpowiedzialny za trening i ewaluację.
- `syn-gen.py` – skrypt generujący syntetyczny zbiór danych oparty na modelowanych atakach wolumetrycznych, pozwalający na wzbogacenie zbioru treningowego i testowego o dodatkowe przypadki niereprezentowane w zbiorach oryginalnych. Posłużył on do wygenerowania trzech dodatkowych syntetycznych zbiorów treningowych, można go użyć w celu generacji kolejnych.
- `test-models.sh` – skrypt powłoki (bash), który automatycznie uruchamia wszystkie skrypty testowe znajdujące się w folderze `TestCases/`. Dla każdego modelu skrypt wykonuje trening, ewaluację, a następnie zapisuje istotne metryki (accuracy, precision, recall, F1 score) do wspólnego pliku wynikowego.

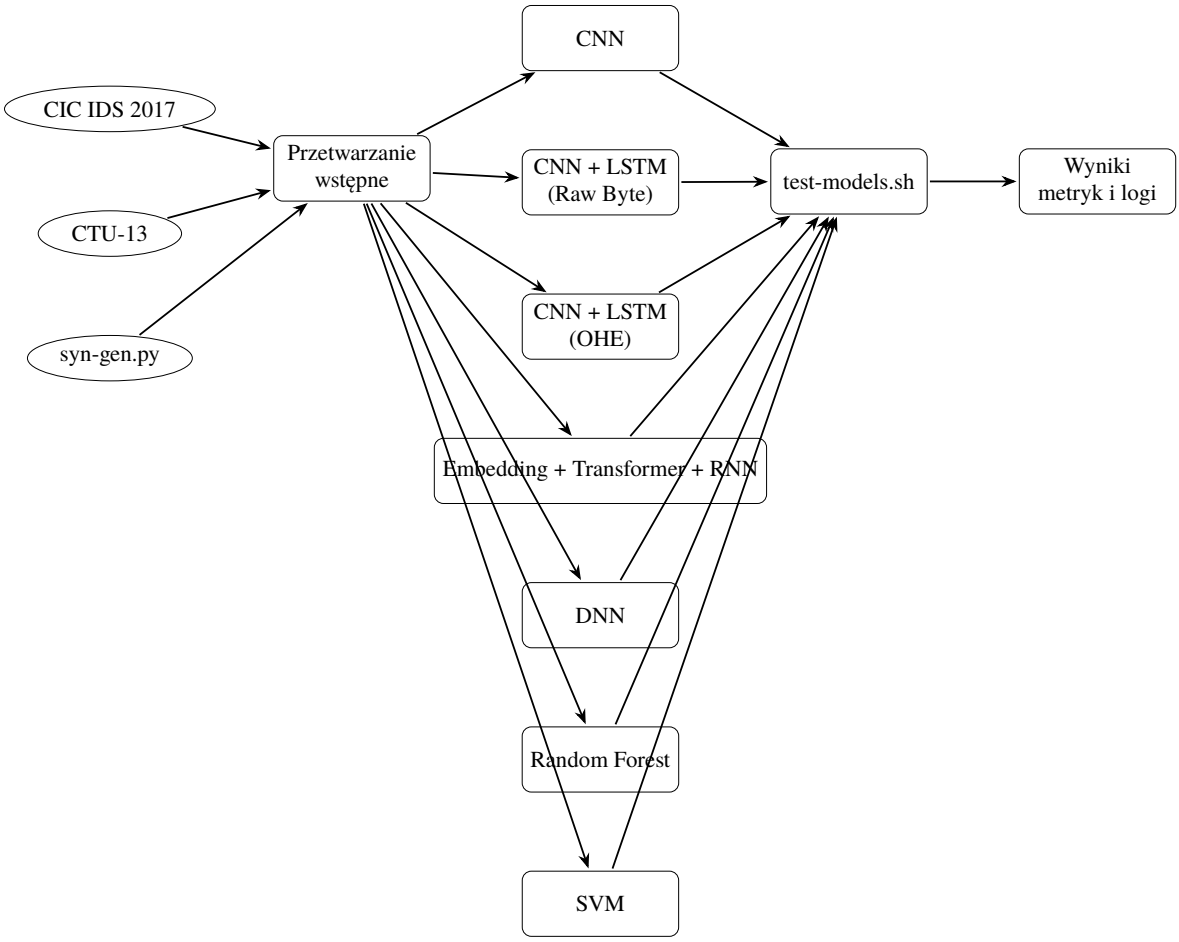
- `setupenv.sh` – skrypt powłoki (bash), który automatycznie generuje wymaganą strukturę folderów dla modeli oraz ustawia środowisko pythona wraz z bibliotekami potrzebnymi do uruchomienia modeli. Skrypt zakłada, że interpretator pythona wraz z pip i venv został już zainstalowany na systemie i wykorzystuje je do ustawiania.

- `requirements.txt` – plik tekstowy wykorzystywany przez pip, zawierający nazwy wszystkich potrzebnych bibliotek do zainstalowania

Każdy z modeli implementowany jest jako osobny plik `.py`, który zawiera definicję architektury sieci, preprocessing danych, a także logikę trenowania i ewaluacji. Po uruchomieniu danego programu, dane wejściowe są wczytywane i przekształcane zgodnie z wymaganym formatem (np. surowa tablica bajtów, one-hot encoding, embedding). Następnie model jest trenowany z wykorzystaniem wybranego zbioru danych, a uzyskane wyniki są zapisywane i wyświetlane użytkownikowi. Skrypt kończy się wypisaniem dystrybucji klas w zbiorze oraz wszystkich najważniejszych metryk wydajności.

Skrypt `test-models.sh` odgrywa kluczową rolę w zapewnieniu powtarzalności i automatyzacji eksperymentów. Po jego uruchomieniu wszystkie przypadki testowe są sekwencyjnie wykonywane, a wyniki są gromadzone w jednym miejscu, co umożliwia łatwe porównanie skuteczności różnych podejść. Skrypt umożliwia też monitorowanie czasu treningu każdego modelu.

Diagram przedstawiony na Rysunku 3.3 ilustruje ogólny schemat przepływu danych oraz interakcję między modułami systemu:



Rysunek 3.3: Schemat przepływu danych i modułów w systemie eksperymentalnym

3.3. Opis środowiska programu potrzebnego do uruchomienia

Do uruchomienia modeli konieczna będzie instalacja interpretera Pythona, wraz z towarzyszącymi mu narzędziami jak pip do instalacji bibliotek czy $venv$ do tworzenia wirtualnych środowisk. Warto zauważyć, że korzystanie z najnowszych wersji Pythona niesie pewne ryzyko w instalacji bibliotek z pip, gdyż nie wszystkie biblioteki od razu dopasowują się poprzez aktualizacje do obsługi najnowszej wersji interpretera. Jest to problem, który napotkałem podczas instalacji biblioteki $tensorflow$, który wymusił na mnie instalacje starszej wersji Pythona (3.12), po to aby móc w ogóle zainstalować tą bibliotekę. Aby zainstalować wszystkie potrzebne biblioteki należy wykonać komendę pip na pliku requirements.txt dołączonym razem z kodem.

Ponieważ programy są napisane w środowisku Python, który jest cross-platform, wybór systemu operacyjnego pozornie wydaje się nieistotny. Warto jednak zauważyć, że moje programy wykorzystują wersje biblioteki tensorflow ze wsparciem dla GPU (kart graficznych), które wymagają od systemu odpowiedniego wsparcia sterowników, umożliwiająca tą komunikacje z GPU. Ponieważ biblioteka tensorflowgpu domyślnie wspiera tylko i wyłącznie karty graficzne od NVIDIA, wymaganiem programów jest uruchamianie ich na komputerze posiadającymi karty NVIDIA obsługującą technologie CUDA (czyli karty z rodziny Maxwell i nowsze). Odpada więc uruchomienie moich programów na systemach operacyjnych MacOS, które nie wspierają środowiska CUDA i kart NVIDIA.

Dla bezproblemowego działania programów zalecana jest jak najnowsza karta NVIDII, jednakże z własnych obserwacji mogę stwierdzić, że karta z rodziny Turing (konkretnie 1660 Ti) powinna już być wystarczająca przy obecnych wersjach bibliotek tensorflow i CUDA. Jednakże warto jest wybrać karty z rodziny co najmniej Turing (pomijając karty 1660 / 1650 i ich warianty) z serii RTX, które mają sprzętowe wsparcie dla sprzętowych jednostek do akceleracji AI, co może potencjalnie przyśpieszyć ćwiczenie modelów.

Należy zainstalować jak najnowsze stabilne sterowniki dostępne dla wybranego modelu karty NVIDIA wraz z obsługa dla środowiska CUDA i biblioteki CUDNN (także najnowsze możliwie wersje dla wybranej karty NVIDIA). W przypadku wybrania systemów Windows jako systemu operacyjnego, zalecane jest wybranie najnowszych sterowników opatrzonych skrótem WHQL, gwarantującym zweryfikowanie stabilności przez firmę Microsoft.

Programy Pythona dołączone do tej pracy należy uruchamiać w wirtualnym środowisku Pythona (venv), w celu izolacji wersji bibliotek w tym projekcie a potencjalnie obecnie zainstalowanych na uruchomionym systemie. W tym wirtualnym środowisku należy także pobierać biblioteki z pomocą komendą pip, należy się upewnić że wykonuje się komendy instalujące z poziomu tego wirtualnego środowiska, a nie całego systemu operacyjnego.

Należy także zainstalować powłokę $bash$ (wraz z wsparciem narzędzia git) jeżeli korzysta się z systemu Windows, aby móc uruchomić dołączone skrypty pomocnicze napisane w bash. Na systemach Uniksowych domyślna powłoka powinna być wystarczająca do uruchomienia skryptów.

Ponieważ programy realizujące modele są napisane w języku Python nie jest konieczny proces budowania plików wyjściowych - wystarczy ustawić poprawnie środowisko i uruchomić z niego skrypty Pythona w interpreterze.

Zbiory danych, na których ćwiczony jest model, czyli CIC IDS 2017 oraz CTU-13 wymagają ręcznego pobrania ze należytych stron internetowych.

Rozmiar na dysku pełnego, głównego zbioru danych CIC IDS 2017 wynosi aż 43 GB i wymaga ręcznego pobrania ze strony kanadyjskiego instytutu CIC. Przy pobieraniu tego zbioru będzie trzeba podać swoje dane, po czym strona automatycznie przekieruje do katalogu apache z którego można wybrać konkretne zbiory. Do samego treningu wykorzystałem archiwum z plikami csv *MachineLearningCSV.zip* oraz pliki pcap z dni: poniedziałek, środa i piątek, więc innych nie ma potrzeba pobierać, co zmniejszy czas pobierania tych zbiorów, który i tak będzie dosyć długi z powodu bardzo wolnych serwerów instytutu.

Do tego trzeba też się zaopatrzyć w zbiór CTU-13, który można pobrać ze strony stratosphereips.org. Należy pobierać fragmenty tego zbioru (próbki 42, 44, 46, 47 oraz 50), które zaopatrują następujące pliki:

- botnet-capture-20110810-neris.pcap
- botnet-capture-20110812-rbot.pcap
- botnet-capture-20110815-fast-flux.pcap
- botnet-capture-20110816-donbot.pcap
- botnet-capture-20110817-bot.pcap

Finalną strukturę katalogów środowiska skompletowaną na podstawie załączonych plików oraz pobranych zbiorów opisuje wyjście komendy tree:

```
licencjat-program
|-- Dataset CTU-13
|   |-- botnet-capture-20110810-neris.pcap
|   |-- botnet-capture-20110812-rbot.pcap
|   |-- botnet-capture-20110815-fast-flux.pcap
|   |-- botnet-capture-20110816-donbot.pcap
|   \-- botnet-capture-20110817-bot.pcap
|-- Dataset CIC IDS 2017
|   |-- Friday-WorkingHours.pcap
|   |-- MachineLearningCVE
|   |   |-- Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
|   |   |-- Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv
|   |   |-- Friday-WorkingHours-Morning.pcap_ISCX.csv
|   |   |-- Monday-WorkingHours.pcap_ISCX.csv
|   |   \-- Wednesday-workingHours.pcap_ISCX.csv
|   |-- Monday-WorkingHours.pcap
|   \-- Wednesday-workingHours.pcap_ISCX.csv
|-- syn-gen.py
|-- synthetic_attack1.pcap
|-- synthetic_attack2.pcap
|-- synthetic_attack3.csv
|-- synthetic_attack3.pcap
|-- TestCase
|   |-- tc11-svm.py
|   |-- tc1-cnn.py
|   |-- tc2-cnn-ltsm.py
|   |-- tc4-embedding-transformer-cnn.py
|   |-- tc6-ohe-cnn-ltsm.py
|   |-- tc8-dnn.py
|   \-- tc9-random-forest.py
| wyniki_modelow_6K.txt
|-- test-models.sh
```

3.4. Wykaz funkcji głównych i pomocniczych wykorzystywanych w programach

Kod realizujący główny model oraz jego warianty podzielony jest na funkcje, które są ponownie używane.

- `read_csv(file_path, num_rows, columns, skip_rows)`

  Wczytuje dane z pliku CSV. Można ograniczyć liczbę wierszy i wybrać kolumny.

- `pcap_to_hex_array(pcap_file, packet_count)`

  Konwertuje plik PCAP do tablicy wartości szesnastkowych.

- `focal_loss(alpha, gamma)`

  Opcjonalna funkcja straty do trenowania na niezbalansowanych danych.

- `build_model(input_shape)`

  Buduje architekturę modelu na podstawie zadanego kształtu wejściowego.

- `train_model(X_train, y_train, X_test, y_test, model, epochs, batch_size)`

  Trenuje model z wykorzystaniem danych treningowych i testowych.

- `extract_labels(labels, col_pos, clear_name)`

  Ekstrahuje etykiety z kolumny na zadanej pozycji, z opcjonalnym czyszczeniem nazw.

- `predict_and_evaluate_model(model, X_train, y_train, X_test, y_test)`

  Przewiduje wyniki i ocenia model na zbiorze testowym i treningowym.

- `load_multiple_datasets(pcap_files, csv_files, packet_count, label_column, label_pos)`

  Ładuje wiele zbiorów danych z plików PCAP i CSV, przygotowując dane i etykiety.

Dla modeli konkurencyjnych względem głównego modelu hybrydowego (CNN+LSTM), wykorzystywane są osobne funkcje pomocnicze, które bazują na cechach ręcznie wybranych z oryginalnych danych CSV. W przeciwieństwie do architektury głębokiego uczenia, gdzie ekstrakcja cech zachodzi automatycznie w warstwach ukrytych, modele klasyczne i bazujące na gotowych cechach (np. Random Forest, SVM, DNN) korzystają z funkcji takich jak `preprocess_data()` i `undersample_data()`, które przygotowują dane wejściowe oraz wspierają proces trenowania i oceny w klasycznych eksperymentach porównawczych.

- `def preprocess_data(df, label_column, top_n_features=TOP_FEATURES)`
  Funkcja dokonuje wstępnego przetwarzania danych: usuwa duplikaty i wartości odstające (`NaN`, `inf`), koduje zmienne kategoryczne metodą one-hot encoding, standaryzuje dane numeryczne, dzieli zbiór na treningowy i testowy oraz wybiera najważniejsze cechy (feature selection) za pomocą klasyfikatora Random Forest.

- `def train_model(X_train, y_train, input_dim)`
  Funkcja trenująca prosty model uczenia maszynowego lub głębokiego uczenia (w zależności od implementacji). Parametr `input_dim` reprezentuje liczbę cech wejściowych. W przypadku klasycznych modeli może zostać zignorowany, natomiast dla modeli opartych na Keras może być wymagany do budowy warstw wejściowych.

- `def predict_and_evaluate(model, X_train, y_train, X_test, y_test)`
  Funkcja generuje predykcje dla zbioru treningowego i testowego, a następnie oblicza metryki jakości klasyfikacji: `accuracy`, `precision`, `recall` i `F1-score`. Dodatkowo wypisywana jest dystrybucja klas w obu zbiorach.

- `def undersample_data(X_train, y_train)`
  Funkcja stosuje technikę `RandomUnderSampler`, która redukuje liczebność klasy dominującej w zbiorze treningowym, aby uzyskać zrównoważony rozkład klas. Zmniejsza to ryzyko uprzywilejowania jednej z klas przez model.

- `def perform_grid_search(X_train, y_train)`
  Opcjonalna funkcja, która wykonuje przeszukiwanie siatki (ang. *grid search*) dla zadanych hiperparametrów modelu, np. w przypadku `RandomForestClassifier` lub `SVM`. Pozwala to znaleźć najlepszą kombinację parametrów na podstawie metryki F1.

3.5. Uruchomienie programu i odczytywanie informacji zwrotnych

Każdy z programów implementujących modele sztucznej inteligencji może zostać uruchomiony bezpośrednio z interpretera Pythona za pomocą polecenia $pythonnazwa_pliku.py$. Jednak w kontekście eksperymentu zaleca się skorzystanie ze specjalnie przygotowanego skryptu, który automatyzuje uruchamianie wszystkich modeli, przeprowadza pomiary wybranych metryk oraz zapisuje wyniki do wspólnego pliku, ułatwiając ich dalszą analizę.

Po uruchomieniu każdy z programów generuje i wypisuje szczegółowe informacje dotyczące metryk wydajności na zbiorach treningowym i testowym, a także prezentuje rozkład klas, co umożliwia ocenę ewentualnego balansu lub niezbalansowania danych. W przypadku sieci neuronowych, program także wypisze swoją architekturę warstw wraz z ilością parametrów w nich oraz pokaże przebieg nauki sieci za pomocą pomiaru metryki dokładności co epokę. Programy nie są w żaden sposób interaktywne i nie wymagają obsługi ze strony testującego, jedynie zwracają pod koniec działania swoje wyniki.

Rozdział 4

Metodologia badań

Celem niniejszej części pracy jest przedstawienie sposobu oceny skuteczności zaprojektowanych modeli detekcji zagrożeń w ruchu sieciowym. Dzięki wykorzystaniu zbiorów danych zawierających etykiety klas, możliwe jest przeprowadzenie pomiarów wydajności z wysoką dokładnością – bez konieczności szacowania lub stosowania przybliżeń. Dla każdego przypadku klasyfikacji znana jest bowiem etykieta rzeczywista, co pozwala jednoznacznie określić, czy decyzja modelu była poprawna. Pozwala to na wyliczenie standardowych metryk ewaluacyjnych, takich jak dokładność (accuracy), precyzja (precision), czułość (recall) oraz miara F1, z pełną kontrolą nad przypadkami prawdziwie i fałszywie sklasyfikowanymi. Taki warunek testowania stanowi podstawę do wiarygodnego porównania wydajności różnych architektur modeli i form reprezentacji danych.

4.1. Pytania pomocnicze

W celu przeprowadzenia rzetelnej analizy skuteczności modelu hybrydowego, zaproponowano zestaw pytań pomocniczych, które umożliwiają szczegółowe rozbicie oceny na komponenty funkcjonalne i architektoniczne. Pytania te zostały sformułowane w taki sposób, aby nie tylko umożliwić bezpośrednie porównanie wydajności z innymi modelami, lecz także zrozumieć wpływ poszczególnych decyzji projektowych – takich jak sposób reprezentacji danych, struktura modelu czy liczność zbioru treningowego – na końcowe wyniki.

- **Jak model hybrydowy wypada na tle pozostałych modeli testowanych w eksperymencie?** Pytanie to pozwala osadzić skuteczność modelu CNN+LSTM w szerszym kontekście porównawczym i ocenić, czy jego wyniki są rzeczywistym postępem względem klasycznych oraz alternatywnych podejść.

- **Jaki wpływ na skuteczność klasyfikacji mają modele oparte na representation learning, w porównaniu do modeli wykorzystujących ręcznie zdefiniowane cechy?** Pozwala to określić, czy automatyczna ekstrakcja cech przez model (np. przez sieci konwolucyjne) faktycznie poprawia zdolność generalizacji oraz redukuje błędy klasyfikacji.

- **Jak modele głębokie oparte na ręcznie dobranych cechach (np. DNN) wypadają w porównaniu do klasycznych metod uczenia maszynowego (np. SVM, Random Forest)?** To pytanie pozwala ocenić, czy korzyści wynikające z głębokiego uczenia są istotne również wtedy, gdy wejściem są cechy inżynierskie, a nie dane surowe.

- **Jaki wpływ na skuteczność modelu mają inne komponenty architektury, takie jak warstwa embedding, mechanizmy transformerowe czy sieci typu ResNet?** Analiza ta odpowiada na

pytanie, czy bardziej zaawansowane mechanizmy reprezentacji i przetwarzania danych mają istotny wpływ na metryki klasyfikacyjne, czy też wprowadzają jedynie nieuzasadnione zwiększenie złożoności.

- **W jaki sposób zmniejszenie lub zwiększenie liczby danych treningowych wpływa na skuteczność modelu?** To pytanie bada odporność modelu na ograniczenia ilościowe zbioru danych, co jest istotne z perspektywy praktycznego wdrażania systemów w środowiskach o ograniczonym dostępie do danych oznaczonych.

4.2. Metryki wykorzystane do pomiarów wydajności

W celu oceny skuteczności działania modeli klasyfikacyjnych zastosowano cztery standardowe metryki: **accuracy**, **precision**, **recall** oraz **F1 score**. Każda z nich odnosi się do innego aspektu działania modelu i pozwala w zróżnicowany sposób spojrzeć na jakość jego predykcji.

- **Accuracy (dokładność)** – informuje, jaki procent wszystkich przewidywań był poprawny. Jest najbardziej użyteczna w przypadku zrównoważonych danych.
- **Precision (precyzja)** – mierzy, jaki procent przypadków zaklasyfikowanych jako pozytywne był faktycznie pozytywny. Jest szczególnie istotna, gdy koszt fałszywie pozytywnych przypadków jest wysoki.
- **Recall (czułość)** – wskazuje, jaki procent rzeczywistych przykładów pozytywnych został poprawnie wykryty przez model. Jest kluczowa, gdy zależy nam na wykryciu jak największej liczby przypadków pozytywnych.
- **F1 score** – średnia harmoniczna precision i recall, stanowiąca zbalansowaną miarę dokładności. Sprawdza się zwłaszcza w przypadku niezbalansowanych zbiorów danych.

Poniżej przedstawiono wzory matematyczne dla każdej z metryk:

$$\textbf{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$
$$\textbf{Precision} = \frac{TP}{TP + FP}$$
$$\textbf{Recall} = \frac{TP}{TP + FN}$$
$$\textbf{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

gdzie:

- TP oznacza True Positives, czyli poprawnie zaklasyfikowane przypadki ataków,
- TN oznacza True Negatives, czyli poprawnie zaklasyfikowane przypadki normalnego ruchu sieciowego,
- FP oznacza False Positives, czyli niepoprawnie zaklasyfikowane przypadki normalnego ruchu sieciowego jako ataki (fałszywe alarmy),
- FN oznacza False Negatives, czyli niepoprawnie zaklasyfikowane przypadki ataków jako normalny ruch sieciowy (niewykrte zagrożenie)

4.3. Warunki testowe

Środowisko testowe wykorzystane do przeprowadzenia eksperymentów oparto na standardowym komputerze osobistym wykorzystywanym przeze mnie do codziennej pracy. Testy przeprowadzono na systemie Arch Linux z domyślnym jądrem w wersji 6.14.2, z wykorzystaniem interpretera Python 3.12, wraz z zainstalowanymi narzędziami pip i venv. Konfigurację sprzętową stanowił ośmiordzeniowy procesor AMD Ryzen 5 5700X (architektura Zen 3) wspierany przez 24 GB pamięci RAM typu DDR4 bez dual channel. Do akceleracji obliczeń w środowisku CUDA wykorzystano kartę graficzną NVIDIA GeForce RTX 4060 Ti (architektura Turing) z 8 GB pamięci VRAM, wyprodukowaną przez firmę Gigabyte. Komputer wyposażono w chłodzenie wodne oraz termopady Honeywell, znane z wysokiej przewodności cieplnej, co zapewnia stabilność działania przy długotrwałym obciążeniu. Testy uruchamiano z dysku SSD SATA o pojemności 256 GB, na partycji NTFS, co może mieć wpływ na czas dostępu do danych przy dużych operacjach wejścia-wyjścia jakim jest wczytywanie zbiorów danych.

Aby przeprowadzić testy wszystkich modeli sztucznej inteligencji, wykorzystano skrypt napisany w języku Bash, którego zadaniem było sekwencyjne uruchamianie każdego z modeli oraz automatyczne zbieranie metryk wydajności w celu ich późniejszego porównania.

Rozdział 5

Wyniki eksperymentów

Testy różnych modeli zostały przeprowadzone na platformie testowej, poprzez uruchomienie skryptu $test - models.sh$, który uruchamia każdy z modeli oraz wydobywa z ich wyjścia standardowego dane dotyczące wydajności modeli, tak aby można było je potem ewaluować. Wyniki modelu zebrane na podstawie uruchomienia wyżej wspomnianego skryptu zostaną przedstawione poniżej w postaci dwóch tabel, jedną dla danych testowych 5 oraz drugą dla danych treningowych 5, gdzie dla każdego z badanych modeli zostały zmierzone wcześniej omawiane, następujące cztery metryki: accuracy, precision, recall, F1.

Tabela 5.1: Skuteczność modeli na danych testowych

Model	**Accuracy [%]**	**Precision [%]**	**Recall [%]**	**F1 Score [%]**
CNN	99.9806	99.9911	99.9688	99.9800
CNN + LSTM	97.8856	96.0707	99.7062	97.8547
Embedding + Transformer	99.9892	100.0000	99.9777	99.9889
CNN + LSTM + OHE	51.3408	51.3408	100.0000	67.8479
DNN	95.7745	99.3738	67.8730	80.6568
Random Forest	99.9509	99.8026	99.8190	99.8108
SVM	78.9367	38.1063	99.7697	55.1489

Tabela 5.2: Skuteczność modeli na danych treningowych

Model	**Accuracy [%]**	**Precision [%]**	**Recall [%]**	**F1 Score [%]**
CNN	99.9688	99.9632	99.9744	99.9688
CNN + LSTM	97.9672	96.3383	99.7249	98.0023
Embedding + Transformer	99.9861	100.0000	99.9722	99.9861
CNN + LSTM + OHE	50.9556	50.9556	100.0000	67.5107
DNN	83.9166	99.9092	67.8949	80.8481
Random Forest	99.9301	99.9671	99.8931	99.9301
SVM	87.5375	80.1433	99.8026	88.8991

Najlepsze rezultaty uzyskał model oparty na architekturze **Embedding + Transformer**, osiągając dokładność **99.9892%** i idealną precyzję na danych testowych. Wyniki te potwierdzają pierwszą pomocniczą hipotezę badawczą, zgodnie z którą nowoczesne modele głębokiego uczenia potrafią skutecznie klasyfikować ruch sieciowy przy użyciu surowych danych wejściowych. Wysoka skuteczność sugeruje, że mechanizmy uwagi (attention) oraz reprezentacja wejścia jako bajtów umożliwiły modelowi uchwycenie złożonych wzorców, istotnych dla wykrywania ataków.

Model CNN, bazujący na ekstrakcji cech przestrzennych, również osiągnął bardzo dobre wyniki - **99.9806%** accuracy, co wskazuje, że nawet bez dodatkowych komponentów sekwencyjnych model potrafi rozróżniać pakiety z wysoką skutecznością. Jednakże dodanie warstwy **LSTM** do sieci CNN, mające na celu uwzględnienie zależności czasowych, nie przyniosło spodziewanej poprawy **CNN + LSTM** uzyskał gorsze wyniki we wszystkich metrykach niż sam CNN. Ten rezultat może świadczyć o tym, że struktura pakietów w zadanym zbiorze nie zawiera wystarczająco silnych zależności sekwencyjnych lub że LSTM nie zostało wystarczająco zoptymalizowane. Tym samym, wyniki te nie wspierają w pełni hipotezy mówiącej o konieczności uzupełnienia CNN o mechanizmy rekurencyjne.

Model CNN + LSTM + OHE uzyskał bardzo słabe wyniki: dokładność **51.3408%**, co sugeruje losowe klasyfikowanie, nie lepsze niż rzut monetą. Choć osiągnięto 100% czułości, niska precyzja wskazuje na znaczną liczbę fałszywych alarmów. To zjawisko wspiera hipotezę badawczą, według której nieoptymalna reprezentacja danych (w tym przypadku one-hot encoding bajtów) może uniemożliwić skuteczne uczenie się modelu. Duża rozdzielczość reprezentacji one-hot dla ciągów bajtów skutkuje znaczną redundancją i utratą kontekstu, co prowadzi do przeciążenia warstw LSTM i słabej generalizacji.

Model DNN trenowany na gotowych cechach osiągnął umiarkowanie wysoką dokładność (**95.7745%**), jednak jego recall był znacznie niższy. Oznacza to, że model skutecznie rozpoznawał pakiety nieszkodliwe, lecz często pomijał ataki. Jest to cenna obserwacja, że mimo użycia ograniczonej liczby cech (12), model potrafił osiągnąć wysoką precyzję, ale jego skuteczność była uzależniona od jakości wybranych cech. Wskazuje to na ograniczenia cech konstruowanych ręcznie i potwierdzające, że wybór cech nadal pozostaje istotnym problemem.

Model Random Forest potwierdził swoją skuteczność w klasyfikacji, osiągając wyniki porównywalne z głębokimi sieciami. Jego precyzja i recall były bardzo wysokie, a różnica między wynikami na zbiorach testowym i treningowym – minimalna. Model ten jest przykładem na to, że klasyczne metody uczące mogą być konkurencyjne, jeśli operują na dobrze przygotowanych danych wejściowych i starannie wybranych cechach. Jednocześnie wyniki te pozwalają częściowo potwierdzić hipotezę, że klasyczne modele również są w stanie skutecznie klasyfikować pakiety sieciowe, choć z mniejszą elastycznością.

Z kolei **model SVM** wypadł wyraźnie gorzej – osiągnął dokładność **78.9367%** i bardzo niską precyzję. Co istotne, różnica pomiędzy skutecznością na danych treningowych i testowych sugeruje problem z generalizacją. Możliwą przyczyną jest tu nadmierna czułość modelu na rozkład danych i brak skalowalności w kontekście dużych, binarnych strumieni. To również potwierdza ograniczenia klasycznych algorytmów w kontekście złożonych danych niskopoziomowych.

Analizując wpływ sposobu reprezentacji danych na skuteczność modeli, można zauważyć, że podejścia oparte na *representation learning* – takie jak **CNN**, **CNN + LSTM** czy **Embedding + Transformer** – oferują dużą elastyczność i zdolność do samodzielnego wyodrębniania cech z surowych danych wejściowych. W szczególności model wykorzystujący połączenie **Embedding + Transformers** i rekurencyjnej warstwy **BiLSTM** osiągnął najwyższe wyniki w niemal wszystkich metrykach, co sugeruje wysoką skuteczność automatycznego wykrywania wzorców w bajtowych reprezentacjach pakietów.

Jednak wbrew oczekiwaniom, klasyczne podejścia nie zostały całkowicie zdeklasowane. Model **Random Forest**, wykorzystujący wyłącznie wcześniej wybrane cechy (top 12), osiągnął bardzo zbliżone wyniki do najlepszych modeli głębokiego uczenia – zarówno pod względem accuracy, jak i precision oraz recall (wszystkie na poziomie około 99.8%). Lecz na dobrze wypadający wynik **Random Forest** przypada także gorzej wypadający wynik **model SVM** z dokładnością **78.9367%**. Można dojść do wniosku, że istnieją techniki klasyczne, które potrafią być konkurencyjne z uczeniem głębokim, jednakże eksperyment wykazał istnienie także gorzej sprawujących się modeli, nad którymi triumfuje.

Analiza wyników uzyskanych przy dziesięciokrotnie zmniejszonym zbiorze danych (zmiana parametru PACKET_COUNT z 60000 do 6000 pakietów, patrz na plik wyniki_modelow_6K.txt dołączo-

ny razem z programem) ujawnia pewne różnice w zachowaniu poszczególnych modeli w warunkach ograniczonej ilości danych treningowych. W większości przypadków zauważalny jest delikatny spadek skuteczności, jednak istotny model główny CNN+LSTM na surowych bajtach odnotował spadek celności z około 97.8856% do 88% oraz 10% spadek precyzji, sugerujący negatywny wpływ LSTM na fałszywe alarmy ataków (fałszywe pozytywy) i ogólną skuteczność. Wskazuje to, że warstwa rekurencyjna LSTM potrzebuje znacznie więcej danych do efektywnego uogólniania wzorców czasowych, bo inaczej wyćwiczenie tej warstwy na zbyt małej ilości danych przynosi większe straty niż niezastosowanie jej w ogóle. Co ciekawe lepiej sobie poradził model CNN+LSTM bazujący na OHE (one-hot encoding) który pomimo posiadania bardzo niskich wyników na obu rozmiarach zbiorów danych, wykazał jednak o około 5% wzrost celności, sugerując że model nie skaluje się dobrze z rozmiarami zbiorów.

Wyniki przeprowadzonych testów potwierdzają, że zaawansowane modele głębokiego uczenia (szczególnie te z mechanizmami uwagi i automatycznym uczeniem reprezentacji) nie przewyższają drastycznie klasycznych podejść zarówno pod względem skuteczności, jak i odporności na fałszywe klasyfikacje. Pomocnicze hipotezy zakładające skuteczność uczenia głębokiego na surowych danych zostały potwierdzone. Z kolei obserwacje dotyczące wpływu reprezentacji danych (np. OHE vs embedding) pokazują, że nie każda forma kodowania nadaje się do tego typu problemów, co prowadzi do ważnego wniosku praktycznego, że odpowiedni wybór reprezentacji wejściowej ma kluczowe znaczenie dla skuteczności systemu wykrywającego zagrożenia sieciowe.

Rozdział 6

Podsumowanie

Celem niniejszej pracy było zaprojektowanie, zaimplementowanie oraz przetestowanie hybrydowego modelu sztucznej inteligencji do wykrywania zagrożeń w ruchu sieciowym. Model hybrydowy zakładał połączenie sieci CNN, znanych z dobrego wykrywania zależności przestrzennych wraz z warstwą LSTM w celu identyfikacji zależności czasowych. Postawiona w rozdziale pierwszym hipoteza badawcza zakładała, że połączenie sieci CNN i LSTM – działających na danych w postaci surowej reprezentacji bajtów pakietów – umożliwi skuteczniejszą detekcję zagrożeń niż tradycyjne modele oparte na ręcznie dobranych cechach lub klasyczne techniki uczenia maszynowego.

W rozdziale drugim omówiłem problematykę systemów IDS/IPS, wyjaśniając dlaczego analizuje ich właściwości w kontekście mojego modelu hybrydowego. Poruszyłem istotne problemy systemów IDS jakim są niereprezentatywne zbiory treningowe, problem w wybieraniu jakościowych cech (ang. features) wejściowych oraz problem fałszywych pozytywów. Poczyniłem potem obszerny wstęp tematyczny do tematyki sieci neuronowych, tłumacząc ideę oraz matematykę stojącą za nimi. Szczególnie skupiłem się na tematyce sieci CNN i RNN stanowiącej trzon modelu hybrydowego. Wyjaśniłem sposób wczytywania danych z plików oraz format (encoding, OHE, raw byte array) jaki jest wczytywany do sieci. Wszystkie rozważania teoretyczna uwieńczyłem opisując szczegóły działania modelu hybrydowego jakim jest architektura wszystkich warstw i hiperparametrów modelu.

W rozdziale trzecim opisałem szczegóły implementacji modelu hybrydowego wraz z jego wariantami oraz modelów porównawczych. Wyjaśniłem strukturę plików i potrzebę istnienia każdego z plików. Opisałem jak się uruchamia skrypt służący do testowania wszystkich modeli i zbierania ich wyników w jedne spójne miejsce. Wyjaśniłem jak należy ustawić środowisko testowe aby uruchomić model, jakie wymagania komputera trzeba spełnić oraz jakie wersje bibliotek zostały wykorzystane. Przedstawiłem podział programów na funkcje i wyjaśniłem jakie maja zadanie.

W rozdziale czwartym krótko poruszyłem problem metodologii badań i sformułowania pytań pomocniczych, po których potem oceniałem modele w części eksperymentalnej pracy. Opisałem dokładnie wzory i intuicje stojącą za metrykami do ewaluacji modeli oraz dokładnie opisałem platformę testową w celu łatwiejszej reprodukcji wyników.

W piątym rozdziale zebrałem wszystkie dane wyjściowe z porównywanych modeli i zebrałem je dwie spójne tabelki na podstawie, których dokonałem analizy modeli.

W trakcie pracy napotkano szereg wyzwań, które miały wpływ na realizację projektu. Jednym z kluczowych problemów był brak realistycznych i reprezentatywnych zbiorów danych, co zostało omówione w rozdziale poświęconym problemom z danymi. Dodatkowo, wyzwaniem była niska liczebność przypadków ataków (imbalans klas z tego wynikający) w dostępnych zbiorach oraz trudność w określeniu, które cechy pakietów sieciowych są rzeczywiście istotne dla procesu klasyfikacji.

Pomimo bardzo wysokich wyników skuteczności osiągniętych przez model, w pracy zidentyfikowano również nierozwiązane problemy, które mogą stać się przedmiotem dalszych badań. Jednym z

nich jest kwestia generalizacji modelu do danych pochodzących z innych źródeł oraz jego odporność na nieznane typy ataków (zero-day). Potrzebne są dalsze eksperymenty z wykorzystaniem różnych zbiorów danych, w tym danych syntetycznych i danych pochodzących z ruchu rzeczywistego. Warto również rozważyć integrację dodatkowych technik wykrywania anomalii, takich jak autoenkodery lub metody klastrowania, aby zwiększyć czułość modelu na nierejestrowane wcześniej typy zagrożeń.

Platforma testowa zaprezentowana w tej pracy może zostać rozbudowana na wiele sposobów. Przede wszystkim warto rozszerzyć ją o możliwość testowania w warunkach zbliżonych do rzeczywistych, np. poprzez implementację środowiska testowego opartego na emulacji ruchu sieciowego w czasie rzeczywistym oraz na poprawie czasu działania modelu, aby bardziej nadawał się do wykorzystania w czasie rzeczywistym. Kolejnym kierunkiem rozwoju jest integracja z istniejącymi narzędziami SIEM lub systemami klasy IDS/IPS w celu oceny praktycznej użyteczności modelu w rzeczywistych warunkach infrastrukturalnych. Można także zaimplementować podsieci identyfikujące źródło pochodzenia pakietu na podstawie danych identyfikujących takich jak adresy MAC czy IP i na podstawie tego śledzić ich reputacje lub analizować co jest uważane dla danego źródła za normę i alarmować, gdy z dużą pewnością wzorce ruchu danego źródła odstają, co mogłoby sugerować włamanie się do niego.

Reasumując, zaproponowany hybrydowy model wykrywania zagrożeń spełnił założone cele i osiągnął wysoką skuteczność. Praca ta potwierdziła zasadność podejścia łączącego konwolucyjne i rekurencyjne sieci neuronowe działające na danych w postaci surowych pakietów, a także wskazała istotne wyzwania w zakresie uczenia systemów bezpieczeństwa sieciowego, które mogą stać się przedmiotem przyszłych badań i wdrożeń.

Bibliografia

[1] J. Gu *et al.*, "Recent advances in convolutional neural networks," 2017. Available online: https://arxiv.org/pdf/1512.07108.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. Available online: https://sophieeunajang.wordpress.com/wp-content/uploads/2020/10/lstm.pdf.

[3] Z. Wu *et al.*, "Graph neural networks: A review of methods and applications," *AI Open*, 2021. Available online: https://www.sciencedirect.com/science/article/pii/S2666651021000012.

[4] K. Fakih *et al.*, "A Taxonomy of Network Threats and the Effect of Current Datasets on Intrusion Detection Systems," *IEEE Access*, 2020. Available online: https://ieeexplore.ieee.org/document/9108270.

[5] M. Injadat *et al.*, "Identifying and Benchmarking Key Features for Cyber Intrusion Detection: An Ensemble Approach," *IEEE Access*, 2019. Available online: https://ieeexplore.ieee.org/document/8765305.

[6] T. Parveen *et al.*, "Towards reducing false alarms in network intrusion detection systems with data summarization technique," *Security and Communication Networks*, 2010. Available online: https://onlinelibrary.wiley.com/doi/epdf/10.1002/sec.562.

[7] Y. Goldberg, "A primer on neural network models for natural language processing," *Journal of Artificial Intelligence Research*, vol. 57, pp. 345–420, 2016. Available online: https://www.jair.org/index.php/jair/article/view/11030/26198.

[8] Y. Bengio *et al.*, "An overview on data representation learning: From traditional feature learning to recent deep learning," *Engineering Applications of Artificial Intelligence*, 2016. Available online: https://www.sciencedirect.com/science/article/pii/S2405918816300459.

[9] Z. Zhang *et al.*, "Transformers Meet Small Datasets," *IEEE Access*, 2023. Available online: https://ieeexplore.ieee.org/document/9944625.

[10] M. Ozkan-Okay *et al.*, "A Comprehensive Systematic Literature Review on Intrusion Detection Systems," *IEEE Access*, 2021. Available online: https://ieeexplore.ieee.org/document/9620099.

[11] A. Khraisat *et al.*, "Hybrid Intrusion Detection System Based on the Stacking Ensemble of C5 Decision Tree Classifier and One Class Support Vector Machine," *Electronics*, 2020. Available online: https://doi.org/10.3390/electronics9010173.

[12] M. Bilal *et al.*, "Auto-Prep: Efficient and Automated Data Preprocessing Pipeline," *IEEE Access*, 2022. Available online: https://doi.org/10.1109/ACCESS.2022.3198662.

[13] Wireshark Wiki, "Development/LibpcapFileFormat," *Wireshark Wiki*, 2020. Available online: https://wiki.wireshark.org/Development/LibpcapFileFormat.

[14] M. Li, "Comprehensive Review of Backpropagation Neural Networks," *Academic Journal of Science and Technology*, 2024.

[15] D. Rumelhart *et al.*, "Learning representations by back-propagating errors," *Nature*, 1986. Available online: https://doi.org/10.1038/323533a0.

[16] Y. Lecun *et al.*, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 1998. Available online: https://doi.org/10.1109/5.726791.

[17] I. Sutskever, "Training recurrent neural networks," *Doctoral dissertation*, University of Toronto, 2013. Available online: https://utoronto.scholaris.ca/server/api/core/bitstreams/41481df7-a5b5-4a6b-a1fa-3c0119069da0/content.

[18] Y. LeCun *et al.*, "Convolutional networks and applications in vision," *Proceedings of 2010 IEEE International Symposium on Circuits and Systems - ISCAS 2010*, 2010. Available online: https://doi.org/10.1109/iscas.2010.5537907.

[19] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*, 2017. Available online: https://arxiv.org/abs/1412.6980.